information

MDPI

Article

CORAL: A Rank-Memory Search Framework for Multi-Objective Feature Selection

Wei Li [1,2,†], Heming Jia [2,3,*,†] and Chunyu Han [4]

[1] School of Computing and Data Science, Fujian University of Technology, Fuzhou 350118, China; 2241308068@smail.fjut.edu.cn
[2] School of Information Engineering, Sanming University, Sanming 365004, China
[3] Fujian Key Laboratory of Agriculture IoT Application, Sanming University, Sanming 365004, China
[4] School of Electrical and Information Engineering, Fujian University of Technology, Fuzhou 350118, China; 2241908008@smail.fjut.edu.cn
* Correspondence: jiaheming@fjsmu.edu.cn
† These authors contributed equally to this work.

Abstract

High-dimensional feature selection aims to identify compact and discriminative feature subsets from large feature spaces. In multi-objective feature selection (MOFS), this task remains challenging because the search space grows exponentially with dimensionality, and conventional binary evolutionary operators may generate ineffective perturbations in sparse high-dimensional spaces. To address these issues, this paper proposes CORAL, a rank-memory search framework for MOFS. CORAL uses a joint continuous score–cardinality representation to model feature priorities and subset sizes and applies Top-K decoding to obtain binary feature subsets. A rank-memory mechanism is introduced to extract feature occurrence information from elite solutions and guide score-space variation. In addition, elite local refinement and feature-number-stratified environmental selection are used to refine candidate subsets and maintain solutions across different sparsity regions. Experiments on 18 benchmark classification datasets show that CORAL achieves balanced performance in terms of solution-set quality, test classification performance, feature compactness, and computational efficiency. Ablation results further demonstrate the complementary roles of rank memory, elite local refinement, and stratified environmental selection.

Keywords: feature selection; multi-objective feature selection; rank memory; continuous score–cardinality representation; stratified environmental selection

Check for updates

Academic Editor: Francesco Fontanella

Received: 11 May 2026
Revised: 4 June 2026
Accepted: 10 June 2026
Published: 13 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

With the rapid development of data acquisition, storage, and computing technologies, high-dimensional datasets have become increasingly common in bioinformatics, hyperspectral image analysis, industrial fault diagnosis, text mining, and related fields. Such datasets usually contain a large number of redundant, irrelevant, or noisy features, which may increase model training costs, degrade classification performance, and weaken generalization ability [1–5].

Feature selection (FS) is an important data preprocessing technique that aims to identify a compact subset of discriminative features without changing the semantic meaning of the original features. Unlike feature extraction, FS preserves the physical meaning of the selected features, making it valuable for dimensionality reduction, model generalization, and result interpretation [6–8]. From an optimization perspective, FS can be naturally

formulated as a multi-objective feature selection (MOFS) problem, where classification performance and feature-subset size are optimized simultaneously [9,10]. Compared with single-objective methods that aggregate different criteria into a weighted objective, MOFS can obtain a set of non-dominated feature subsets in a single run, offering decision-makers alternative trade-offs between classification performance and subset compactness.

Multi-objective evolutionary algorithms (MOEAs) are widely used in MOFS because they can approximate multiple trade-off solutions without requiring differentiable objectives or convex search spaces. Representative MOEAs include the non-dominated sorting genetic algorithm II (NSGA-II) and the multi-objective evolutionary algorithm based on decomposition (MOEA/D) [11–13]. However, when applied to high-dimensional or ultra-high-dimensional feature spaces, existing MOEAs still face clear limitations in search efficiency and scalability.

A major difficulty of high-dimensional FS lies in the exponentially expanding combinatorial search space. For a dataset with D features, the number of candidate feature subsets can reach 2^D, making exhaustive search and global optimal subset identification impractical [14,15]. When D reaches thousands or tens of thousands, high-quality feature subsets are usually sparsely distributed in the search space. Traditional binary encodings and their associated operators, such as bit-flip mutation and conventional crossover, often provide only coarse perturbations and may fail to capture the sparse structure of high-dimensional FS. Consequently, they may generate many ineffective or low-quality candidate solutions, reducing the overall search efficiency. The problem is further complicated by feature redundancy, duplicate solutions, and feature interactions. Selecting features only according to individual importance may ignore complementary feature combinations or repeatedly introduce highly correlated redundant features, thereby affecting classification performance, subset compactness, and population diversity [16,17].

Maintaining an effective balance between convergence pressure and population diversity is also challenging in high-dimensional MOFS. In such scenarios, many candidate solutions may become mutually non-dominated, which weakens the discriminative power of Pareto-dominance-based selection mechanisms and slows the approximation to the true Pareto front. Recent studies have attempted to improve high-dimensional MOFS through search-space reduction, fast dimensionality reduction, evolutionary multitasking, feature-weight-guided search, learning-assisted search, redundant-solution handling, and adaptive evolutionary strategies [10,13,18–21]. Although these methods improve scalability to some extent, many of them still rely on fixed priors, static search spaces, or traditional binary operators. As a result, it remains difficult to consistently balance feature compression, classification performance, and solution-set distribution quality across datasets with different dimensionalities.

To address these issues, this paper proposes CORAL, a rank-memory search framework for multi-objective feature selection. Instead of directly performing random crossover and bit-flip mutation in the original binary space, CORAL models candidate feature subsets through a joint continuous score–cardinality representation. The continuous score vector describes the relative priority of features, while the cardinality variable explicitly controls the subset size. A Top-K decoding mechanism then maps the continuous representation into binary feature subsets, establishing a stable connection between continuous search and discrete feature selection.

CORAL further introduces a rank-memory-guided search mechanism to exploit historical information accumulated during evolution. Specifically, feature occurrence frequencies are extracted from elite individuals and incorporated as a memory bias into the continuous search process. This mechanism introduces a feature-preference bias into score-space variation, so that features repeatedly selected by elite solutions are more likely to be explored.

In addition, an elite local refinement strategy is designed to adjust marginal features in elite solutions through lightweight remove–add operations, while stratified environmental selection maintains solution distributions across different sparsity regions. These mechanisms are designed to support a stable trade-off among feature compression, classification performance, and Pareto solution-set distribution.

The novelty of CORAL lies in the joint design of score-space evolution, explicit subset-size control, rank-memory-guided priority updating, and sparsity-aware population maintenance. Existing score- or weight-based feature selection methods usually use feature scores as auxiliary ranking, initialization, or filtering information. In contrast, CORAL treats the score vector as an evolvable feature-priority representation and couples it with an explicit cardinality variable. Thus, feature-priority updating and subset-size control are handled separately and connected through deterministic Top-K decoding.

In addition, the rank memory in CORAL is embedded into the continuous score-space mutation equation, rather than being used only to bias binary bit-flip probabilities, update an archive, or preselect candidate features. Historical feature-occurrence information therefore guides the evolution of feature priorities while still allowing objective evaluation and environmental selection to reject redundant or ineffective features. The feature-number-stratified environmental selection further maintains candidate solutions across different sparsity regions, reducing the risk of population concentration in a narrow feature-size range.

The main contributions of this paper are summarized as follows:

1. A joint continuous score–cardinality representation is developed for high-dimensional MOFS, where the score vector evolves feature priorities, and the cardinality variable explicitly controls subset size through Top-K decoding.
2. A rank-memory-guided score-space search operator is proposed, in which elite feature-occurrence information is embedded into continuous mutation to guide feature-priority evolution.
3. A sparsity-aware solution maintenance strategy is designed by combining elite local refinement with feature-number-stratified environmental selection, so that candidate solutions are maintained across different sparsity regions.

The remainder of this paper is organized as follows. Section 2 reviews related studies on MOFS and high-dimensional evolutionary search. Section 3 describes the proposed CORAL framework in detail. Section 4 presents the experimental settings and compared algorithms. Section 5 analyzes and discusses the experimental results. Section 6 concludes this paper and outlines future research directions.

2. Related Work

2.1. Formulation of Multi-Objective Feature Selection

In many real-world optimization tasks, multiple objectives need to be optimized simultaneously, and these objectives are often conflicting [22]. For a minimization multi-objective optimization problem (MOP) with m objectives, the general formulation can be written as

$$\begin{aligned} \min \quad & \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_m(\mathbf{x}))^T \\ \text{s.t.} \quad & \mathbf{x} \in \Omega, \end{aligned} \tag{1}$$

where $\mathbf{x} = (x_1, x_2, \ldots, x_D)^T$ denotes a D-dimensional decision vector, Ω is the feasible solution space, $f_i(\mathbf{x})$ is the ith objective function, and $\mathbf{F}(\mathbf{x})$ is the objective vector composed of all objective functions. Since different objectives usually cannot be optimized simultaneously to their individual optima, the core task of multi-objective optimization is not to

find a single optimal solution but to obtain a set of non-dominated solutions that reflect different trade-offs among the objectives.

In feature selection, a candidate feature subset is usually represented by a binary vector, where each dimension indicates whether the corresponding feature is selected [10,23]:

$$\begin{aligned} &\mathbf{x} = (x_1, x_2, \ldots, x_D)^T, \\ &x_j \in \{0,1\}, \quad j = 1, 2, \ldots, D, \end{aligned} \tag{2}$$

where D denotes the dimensionality of the original feature space. If $x_j = 1$, the jth feature is selected; otherwise, if $x_j = 0$, the feature is removed. Therefore, feature selection essentially searches among 2^D candidate subsets for feature combinations with good classification performance and small subset size. When D is large, the search space expands exponentially, making exhaustive search computationally infeasible. Thus, heuristic or evolutionary search methods are usually required.

For classification tasks, MOFS usually minimizes the selected-feature ratio and the classification error rate simultaneously, aiming to obtain Pareto trade-offs between subset compactness and classification performance [9,13]. Accordingly, MOFS can be formulated as the following bi-objective minimization problem:

$$\begin{aligned} \min \quad & \mathbf{F}(\mathbf{x}) = \left(f_{\text{ratio}}(\mathbf{x}), f_{\text{err}}(\mathbf{x})\right)^T \\ \text{s.t.} \quad & \mathbf{x} \in \{0,1\}^D, \end{aligned} \tag{3}$$

where $f_{\text{ratio}}(\mathbf{x})$ denotes the ratio of selected features, and $f_{\text{err}}(\mathbf{x})$ denotes the cross-validation classification error rate obtained using the candidate feature subset $\mathbf{x}$. They are defined as

$$\begin{cases} f_{\text{ratio}}(\mathbf{x}) = \dfrac{\sum_{j=1}^{D} x_j}{D}, \\ f_{\text{err}}(\mathbf{x}) = 1 - \text{Acc}_{\text{CV}}(\mathbf{x}), \end{cases} \tag{4}$$

where $\text{Acc}_{\text{CV}}(\mathbf{x})$ denotes the cross-validation classification accuracy obtained using the candidate feature subset $\mathbf{x}$. A smaller $f_{\text{ratio}}(\mathbf{x})$ indicates a more compact feature subset, while a smaller $f_{\text{err}}(\mathbf{x})$ indicates better classification performance. Since classification performance and feature-subset size are usually conflicting, the goal of MOFS is to obtain a set of non-dominated solutions with different trade-offs between these two objectives.

To describe the relationships among solutions in multi-objective optimization, the Pareto dominance relation is commonly introduced. For any two feasible solutions $\mathbf{u}, \mathbf{v} \in \Omega$, $\mathbf{u}$ is said to dominate $\mathbf{v}$, denoted by $\mathbf{u} \prec \mathbf{v}$, if and only if

$$\left(\forall i \in \{1, 2, \ldots, m\},\ f_i(\mathbf{u}) \leq f_i(\mathbf{v})\right) \wedge \left(\exists j \in \{1, 2, \ldots, m\},\ f_j(\mathbf{u}) < f_j(\mathbf{v})\right). \tag{5}$$

If a feasible solution is not dominated by any other feasible solution, it is called a Pareto-optimal solution. All Pareto-optimal solutions in the decision space form the Pareto set (PS), and their mappings in the objective space form the Pareto front (PF), which are defined as

$$\text{PS} = \{\mathbf{u} \in \Omega \mid \nexists\, \mathbf{v} \in \Omega,\ \mathbf{v} \prec \mathbf{u}\}, \tag{6}$$

and

$$\text{PF} = \{\mathbf{F}(\mathbf{u}) \mid \mathbf{u} \in \text{PS}\}. \tag{7}$$

Therefore, for a multi-objective feature selection algorithm, the optimization goal is not merely to obtain a single solution with high classification performance or a small number of features. Instead, the algorithm should approximate the true Pareto front as closely as

possible while maintaining good convergence, distribution, and diversity of the obtained solution set.

2.2. Evolutionary Algorithms for High-Dimensional Feature Selection

Feature selection methods are commonly categorized into filter, wrapper, and embedded methods [3,7,24,25]. Filter methods select features according to statistical properties, correlations, or information measures, such as information gain, Chi-squared test, Fisher score, mRMR, ReliefF, odds ratio, Gini index, and bi-normal separation [26,27]. They are computationally efficient and suitable for fast preprocessing, but their performance is sensitive to ranking criteria, threshold settings, and data-distribution skewness. Wrapper methods evaluate candidate subsets by training a classifier and usually show stronger task adaptability, but their computational cost becomes high in large feature spaces [24,28]. Embedded methods incorporate feature selection into model training and provide a compromise between efficiency and performance, although their effectiveness is often tied to specific model structures and assumptions.

To combine the computational efficiency of filter methods with the task adaptability of wrapper methods, hybrid filter–wrapper feature selection methods have also attracted attention. These methods usually first generate candidate feature subsets using independent evaluation criteria, distance measures, or feature-ranking mechanisms and then select the final feature combination from the candidate subsets using a specific classifier and cross-validation [28,29]. Such methods can alleviate the high computational cost of pure wrapper methods to some extent. However, their search processes still rely on predefined ranking criteria or local incremental search strategies, making it difficult to sufficiently explore the high-dimensional combinatorial space.

Because FS is discrete, combinatorial, and nonconvex, evolutionary computation methods have been widely adopted due to their global search capability and adaptability to black-box evaluation functions [25,30,31]. Early studies often formulated feature selection as a single-objective optimization problem, in which classification error and the number of selected features were aggregated into a scalar objective through weighted summation or penalty terms. These methods are easy to implement and often converge efficiently on low- and medium-dimensional datasets. However, single-objective modeling depends on predefined weights or penalty parameters, making it difficult to adapt to changes in objective preferences across different datasets. Moreover, these methods usually output only one final solution and cannot provide decision-makers with diverse trade-off solutions between different sparsity levels and classification performance.

In contrast, multi-objective evolutionary algorithms can search for multiple non-dominated feature subsets in a single run, making them more suitable for handling the conflict between classification performance and subset size in feature selection. Early studies explicitly modeled feature selection as a bi-objective optimization problem that simultaneously minimizes the number of selected features and the classification error rate and employed NSGA-II to search for a set of Pareto-optimal feature subsets [32]. Representative multi-objective evolutionary algorithms include NSGA-II and MOEA/D. NSGA-II maintains convergence and diversity through non-dominated sorting and crowding distance, whereas MOEA/D decomposes a multi-objective problem into a set of scalar subproblems and performs parallel search through neighborhood cooperation [11,12]. These general frameworks provide an effective basis for MOFS. However, they were not originally designed for high-dimensional sparse feature spaces and still exhibit clear limitations when dealing with large-scale feature selection problems.

The main difficulty of high-dimensional feature selection lies in its extremely large search space and the sparse distribution of high-quality solutions. As feature dimensional-

ity increases, crossover and mutation operators under traditional binary encoding tend to generate many ineffective perturbations, making it difficult to continuously produce compact and discriminative feature subsets. Meanwhile, high-dimensional data often contain a large number of redundant features and complex feature interactions. As a result, evolutionary processes driven only by random perturbation or a single selection pressure may easily stagnate. In addition, in high-dimensional discrete decision spaces, many candidate solutions may show similar performance in terms of classification error and feature number. This makes it difficult for Pareto dominance to sufficiently distinguish solution quality, thereby weakening the selection pressure that guides the population toward high-quality front regions.

To address these issues, recent studies on high-dimensional MOFS have mainly improved the optimization process from three aspects: search space reduction, sparse search bias, and population diversity maintenance. Search-space reduction methods usually use filter criteria, feature ranking, or multi-stage compression strategies to remove some redundant or irrelevant features before evolutionary search, thereby reducing the difficulty of subsequent optimization [10,13,19]. These methods can significantly reduce computational cost. However, if the prescreening process relies excessively on single-feature evaluation, it may mistakenly remove key features that are useful only when combined with other complementary features. Sparse-search methods encourage algorithms to search for compact feature subsets through sparse initialization, sparse mutation, or nonzero-variable guidance. Nevertheless, overly strong sparsity bias may weaken the exploration of effective feature combinations. Diversity-maintenance methods alleviate population degeneration through redundant-solution repair, duplicate-solution handling, or adaptive operator adjustment, thereby enhancing search stability and solution-set distribution. In addition, multiform optimization has also been introduced into MOFS. MFFS constructs auxiliary single-objective feature selection tasks and exchanges information among different search directions to improve the search efficiency and solution-set diversity of MOFS [33]. Furthermore, MFMOEA uses multiple auxiliary single-objective tasks, knowledge-transfer strategies, and diversity-enhancement mechanisms to guide the MOFS task toward more promising decision regions and accelerate convergence [34]. However, these methods are still usually based on binary feature-subset search, and their ability to characterize continuous feature priorities and stable historical feature patterns remains limited.

Recently, high-dimensional MOFS has been further extended from different perspectives. Some studies attempt to exploit prior information or multimodal search behavior to preserve alternative feature subsets and improve exploration ability [35]. Feature-interaction-aware methods explicitly model correlations or conditional relationships among features to reduce redundant combinations and improve offspring generation [16]. In addition, learning-assisted feature selection has been investigated by combining radiomics or deep features with multi-objective optimization for compact feature subset construction [36]. Multi-label feature selection has also been studied under lightweight multi-objective formulations to balance feature relevance, redundancy, and label-related criteria [37]. These studies show that recent MOFS research is moving toward prior-guided, interaction-aware, and learning-assisted search. Different from these methods, CORAL does not rely on deep feature extractors, explicit feature-interaction probability modeling, or irreversible search-space pruning. Instead, it embeds rank-memory guidance into an evolvable continuous score–cardinality representation, aiming to provide lightweight historical guidance while keeping the original feature space accessible.

Overall, existing high-dimensional MOFS methods have improved search-space reduction, sparsity modeling, redundancy handling, and adaptive search. Nevertheless, many methods still perform variation mainly in the binary feature-subset space, where

bit-level perturbations may be inefficient in sparse high-dimensional settings. In addition, historical information from elite solutions is often used only indirectly, and its role in guiding feature-priority updates remains underexplored.

These observations motivate the design of CORAL. Although recent high-dimensional MOFS studies have explored feature weighting, memory guidance, search-space reduction, and sparse evolutionary search, many of them still use feature scores or historical information as auxiliary guidance outside the main search representation or perform variation mainly in the binary subset space. CORAL instead uses the score vector as the evolvable representation, couples it with an explicit cardinality variable, and embeds rank memory into continuous score-space mutation. This design provides a lightweight way to guide feature-priority evolution while preserving the original feature space and maintaining solutions across different sparsity levels.

3. Proposed Method

This section presents CORAL, the proposed rank-memory search framework for high-dimensional multi-objective feature selection. CORAL characterizes feature priorities in a continuous score space and guides the search using feature occurrence information accumulated from elite solutions. The framework integrates feature-priority modeling, subset-size control, and sparsity-aware solution maintenance within a unified evolutionary search process. The overall framework of CORAL is illustrated in Figure 1, and its general procedure is given in Algorithm 1.

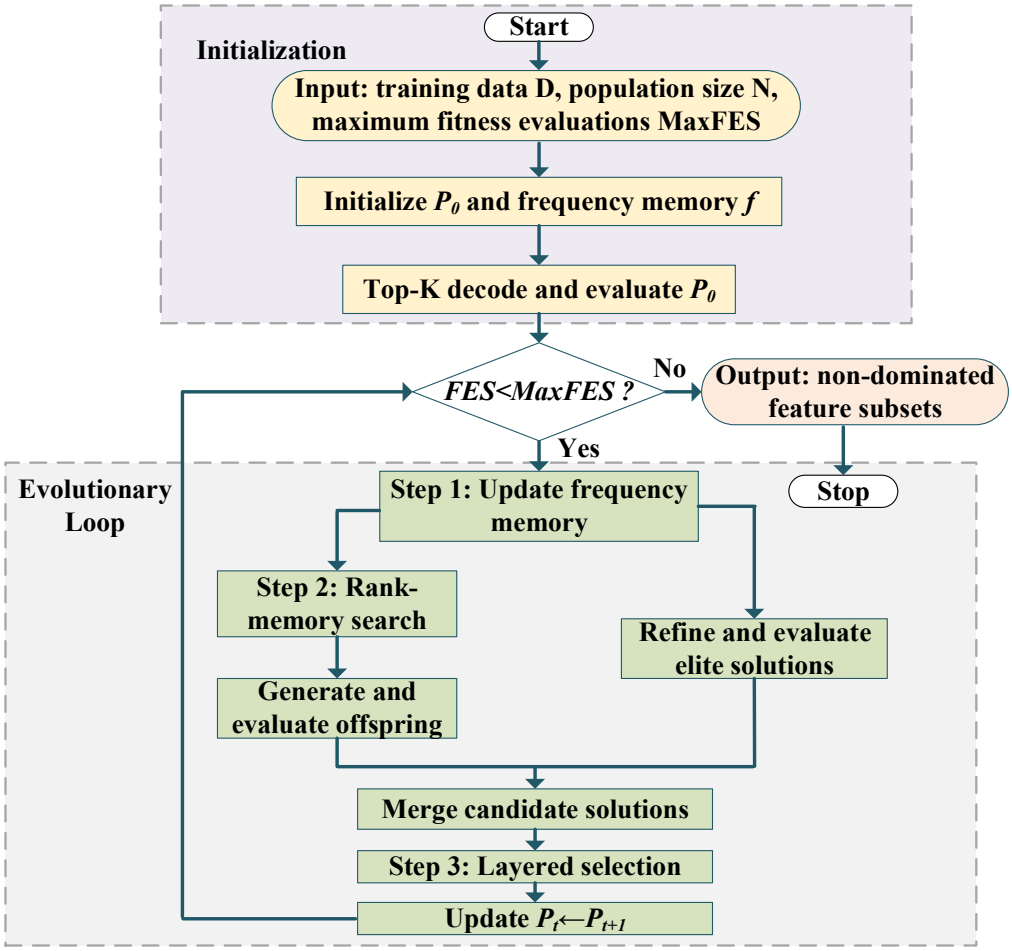


Figure 1. Overall framework of CORAL.

Algorithm 1: CORAL: General Framework

Input: Dataset $\mathcal{D}$; population size N; maximum number of fitness evaluations MaxFES; number of sparsity layers B; number of refined elites N_m

Output: Non-dominated solution set NDSet

Initialization: Generate the initial population $P_0 = \{\langle \mathbf{s}_i, k_i \rangle\}_{i=1}^{N}$ and initialize the frequency memory $\mathbf{f}$

Decode P_0 by Top-K mapping and evaluate the initial population

while FES $<$ MaxFES **do**

  $(P_t^o, \mathbf{f}) \leftarrow \text{RankMemorySearch}(P_t, \mathbf{f})$

  $P_t^m \leftarrow \text{EliteRefinement}(P_t, \mathbf{f}, N_m)$

  $\mathcal{C}_t \leftarrow P_t \cup P_t^o \cup P_t^m$

  $P_{t+1} \leftarrow \text{StratifiedSelection}(\mathcal{C}_t, N, B)$

NDSet $\leftarrow \text{Nondominated}(P_t)$

return NDSet

CORAL consists of three main modules: joint continuous score–cardinality representation, rank-memory-guided search, and stratified environmental selection. The joint continuous score–cardinality representation provides a stable search representation; the rank-memory mechanism exploits feature-preference information from elite solutions; and stratified environmental selection maintains the distribution of solutions across different feature-size regions. These key mechanisms are described in detail below.

The three modules are designed to affect different aspects of the search process. The score–cardinality representation separates feature-priority updating from subset-size control, so that score-space variation can adjust feature rankings without necessarily changing the sparsity level. The rank-memory update smooths elite feature-occurrence information across generations and introduces it as a soft bias in score-space mutation. Thus, historical feature preferences can guide the search direction without deterministically fixing selected features. Stratified environmental selection limits the number of retained solutions in each feature-number interval, reducing over-concentration in a narrow sparsity region. The effects of these mechanisms are examined empirically through HV/IGD results, convergence curves, Pareto-front visualizations, and ablation studies.

3.1. Joint Continuous Score–Cardinality Representation and Top-K Decoding

High-dimensional feature selection is essentially a discrete binary combinatorial optimization problem. Traditional genetic algorithms and binary evolutionary algorithms usually use 0–1 encoding to represent whether each feature is selected. However, this representation often makes the search process behave like random feature recombination and has limited ability to characterize the continuous contribution differences among features. Existing studies have shown that real-valued encoding or feature-weight vectors can characterize feature contribution differences to some extent, thereby providing more fine-grained search information for high-dimensional feature selection [38]. Unlike methods that directly optimize feature weights in a classifier-driven manner, CORAL jointly models a continuous score vector and an explicit cardinality variable to simultaneously control feature priorities and candidate subset sizes.

For the ith individual, its representation is defined as

$$A_i = \langle \mathbf{s}_i, k_i \rangle, \tag{8}$$

where $\mathbf{s}_i = [s_{i,1}, s_{i,2}, \ldots, s_{i,D}] \in \mathbb{R}^D$ is a continuous score vector representing the relative priorities of features, and k_i is the target feature-subset size controlling the number of

activated features. This representation decouples feature ranking from subset-size control, enabling the search process to smoothly adjust feature priorities in the continuous score space while explicitly constraining the sparsity level of candidate solutions.

This separation is particularly useful for sparse high-dimensional feature selection. In a binary representation, changing selected features and changing subset size are often coupled, because bit-level perturbations may simultaneously alter feature identities and the number of selected features. In CORAL, score-space variation mainly updates the relative priorities of features, while the cardinality variable determines how many top-ranked features are activated. Therefore, the algorithm can explore alternative feature rankings without unintentionally changing the sparsity level and can adjust the subset size without rebuilding the feature-priority structure from scratch. This design reduces ineffective structural perturbations and provides a more stable way to construct sparse candidate subsets in large feature spaces.

Since fitness evaluation is still performed on binary feature subsets, CORAL adopts Top-K decoding to map the continuous representation into a binary selection vector. Specifically, the feature scores in $\mathbf{s}_i$ are sorted in descending order, and the top k_i features are selected as activated features. Let $\Omega_{k_i}(\mathbf{s}_i)$ denote the index set of the top k_i features according to the scores in $\mathbf{s}_i$. The corresponding binary vector $\mathbf{x}_i$ is defined as

$$x_{i,j} = \begin{cases} 1, & j \in \Omega_{k_i}(\mathbf{s}_i), \\ 0, & \text{otherwise}. \end{cases} \tag{9}$$

Figure 2 illustrates the joint continuous score–cardinality representation and the Top-K decoding process. The continuous score vector determines the relative ranking of features, while the cardinality variable determines the number of selected features. Together, they specify the final candidate feature subset. Compared with directly performing bit-flip operations in the binary space, this mechanism can reduce ineffective structural perturbations and provide a more stable search representation for subsequent rank-memory-guided search.

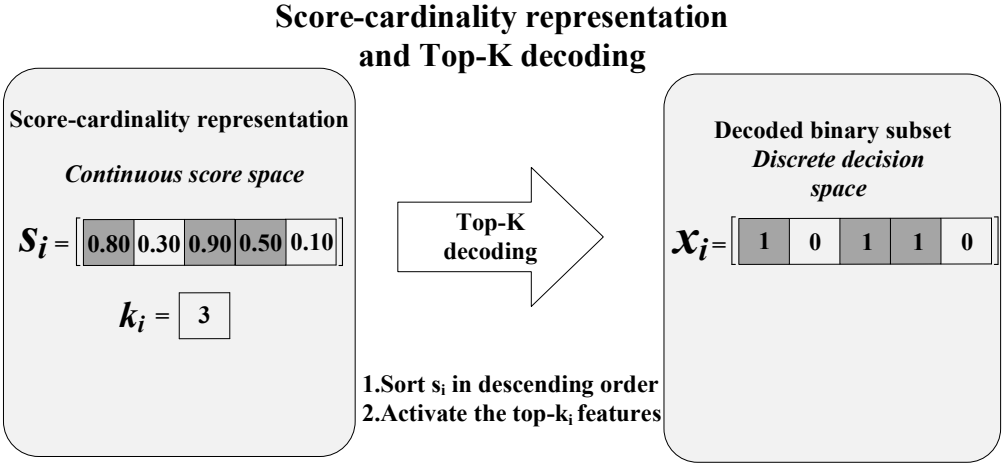


Figure 2. Top-K decoding from the score–cardinality representation. Gray cells indicate selected features, and white cells indicate unselected features.

3.2. Rank-Memory-Guided Search Mechanism

Based on the score–cardinality representation, CORAL introduces rank memory to guide candidate generation in the continuous score space. This mechanism constructs a frequency memory from features repeatedly occurring in elite solutions and incorporates it into score-space variation.

In each generation, CORAL first identifies an elite set $\mathcal{E}^{(t)}$ from the current population according to non-dominated ranking and crowding distance. For each feature j, the current elite occurrence frequency is calculated as

$$q_j^{(t)} = \frac{1}{|\mathcal{E}^{(t)}|} \sum_{\mathbf{x}_i \in \mathcal{E}^{(t)}} x_{i,j}, \tag{10}$$

where $x_{i,j} = 1$ indicates that feature j is selected by the decoded solution $\mathbf{x}_i$, and $x_{i,j} = 0$ otherwise. Thus, $q_j^{(t)} \in [0,1]$ is the normalized proportion of elite solutions containing feature j.

The frequency memory is updated by an exponential smoothing rule:

$$f_j^{(t+1)} = (1-\rho) f_j^{(t)} + \rho q_j^{(t)}, \tag{11}$$

where $f_j^{(t)} \in [0,1]$ is the historical memory value of feature j, and $\rho \in [0,1]$ is the memory update coefficient. The term $1-\rho$ retains historical information, whereas ρ controls the contribution of the current elite frequency. Therefore, a larger ρ makes the memory more responsive to the current elite set, while a smaller ρ produces a smoother update. Since both $f_j^{(t)}$ and $q_j^{(t)}$ are bounded in $[0,1]$, the updated memory $f_j^{(t+1)}$ is also bounded in $[0,1]$. The same ρ value is used for all datasets and independent runs.

The frequency memory is used only as a soft bias in score-space variation, rather than as a deterministic feature-selection rule. A frequently occurring feature may receive a higher score tendency, but it is not forced into the final subset. Each generated candidate is still decoded, evaluated by the two objective functions, and filtered through environmental selection. Therefore, frequently selected but redundant features can be discarded if they do not improve the objective trade-off or solution distribution.

Based on the updated frequency memory, CORAL generates offspring in the continuous score–cardinality space. For each target individual i, three mutually different indices r_1, r_2, and r_3 are randomly sampled from $\{1,2,\ldots,N\}\backslash\{i\}$. The mutant score vector is generated as

$$\mathbf{v}_i = \mathbf{s}_{r_1} + F(\mathbf{s}_{r_2} - \mathbf{s}_{r_3}) + \gamma(2\mathbf{f}^{(t+1)} - \mathbf{1}) + \sigma \boldsymbol{\epsilon}_i, \tag{12}$$

where F is the differential scaling factor, γ controls the strength of rank-memory guidance, and $\boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is a Gaussian perturbation vector. The term $2\mathbf{f}^{(t+1)} - \mathbf{1}$ maps the memory vector from $[0,1]$ to $[-1,1]$. Thus, features frequently appearing in elite solutions receive a positive score bias, whereas rarely selected features receive a negative bias. This memory term acts only as a soft search preference and does not force any feature to be selected.

The cardinality variable is updated by differential perturbation, rounding, and boundary clipping:

$$\tilde{k}_i = \mathrm{clip}_{[K_{\min}, K_{\max}]}(\mathrm{round}(k_{r_1} + \lambda_k(k_{r_2} - k_{r_3}) + \sigma_k \eta_i)), \tag{13}$$

where λ_k is the scaling factor in the cardinality dimension, $\eta_i \sim \mathcal{N}(0,1)$, and σ_k is the perturbation strength. The rounding operation ensures an integer subset size, and the clipping operation keeps the generated cardinality within $[K_{\min}, K_{\max}]$.

Binomial crossover is then performed between the parent score vector $\mathbf{s}_i$ and the mutant score vector $\mathbf{v}_i$:

$$u_{i,j} = \begin{cases} v_{i,j}, & \xi_j < p_c \text{ or } j = j_{\mathrm{rand}}, \\ s_{i,j}, & \text{otherwise.} \end{cases} \tag{14}$$

where $\xi_j \sim U(0,1)$, p_c is the crossover probability, and j_{rand} guarantees that at least one score component is inherited from the mutant vector. The trial score vector is clipped

into $[-10, 10]$ to avoid excessively large magnitudes. The offspring cardinality is set to $\tilde{k}_i$ with probability 0.5; otherwise, it inherits the parent cardinality k_i. Finally, the trial score–cardinality representation $\langle \mathbf{u}_i, k_i^{\text{off}} \rangle$ is decoded into a binary feature subset using the Top-K rule in Equation (9).

In this study, $F = 0.5$, $p_c = 0.9$, $\lambda_k = 0.3$, $\sigma = 0.1$, and $\sigma_k = 0.5$ were fixed implementation constants used to reproduce the score-space search operator. The cardinality bounds were set to $K_{\min} = 1$ and $K_{\max} = \text{round}(0.2D)$. The main CORAL parameters analyzed in the sensitivity study remained B, N_m, ρ, and γ.

To further exploit promising search regions, CORAL applies a lightweight local refinement to N_m elite individuals. The refined elites are selected from the current population according to non-dominated ranking and crowding distance. For an elite individual $A_i = \langle \mathbf{s}_i, k_i \rangle$, let $\mathcal{S}_i$ denote the selected feature set obtained by Top-K decoding, and let $\overline{\mathcal{S}}_i = \{1, 2, \ldots, D\} \backslash \mathcal{S}_i$ denote the corresponding unselected feature set.

The removable feature is selected from the current selected set as the feature with the lowest continuous score:

$$j_i^- = \arg\min_{j \in \mathcal{S}_i} s_{i,j}. \tag{15}$$

Since the selected features are generated by Top-K decoding, j_i^- corresponds to the selected feature with the weakest score support in the current elite subset. Removing this feature causes only a local change around the elite solution rather than destroying the main selected-feature structure.

The insertable feature is selected from the unselected set according to the frequency memory:

$$j_i^+ = \arg\max_{j \in \overline{\mathcal{S}}_i} f_j^{(t+1)}. \tag{16}$$

If multiple unselected features have the same memory value, the feature with the larger current score $s_{i,j}$ is selected. This rule gives priority to features that frequently appear in elite solutions while still considering the current score ranking.

The refined candidate is then constructed by removing j_i^- from $\mathcal{S}_i$ and inserting j_i^+ into it. The cardinality remains unchanged, i.e., $k_i^m = k_i$, so the refinement only adjusts the feature composition around the elite solution without changing its subset size. The corresponding binary vector is constructed by activating the refined selected-feature set and setting all other features to 0.

It should be noted that the memory-based insertion rule does not assume that a high-frequency feature is always useful or complementary to the current subset. The frequency memory only provides a candidate replacement direction. The original elite solution is kept unchanged, and the refined solution is added to the candidate pool as an additional solution rather than directly overwriting the original one. After refinement, the generated candidate is evaluated by the same objective functions as other offspring and then competes with parent, offspring, and other refined solutions in stratified environmental selection. Therefore, a common but redundant high-frequency feature can survive only if the refined subset improves the objective trade-off or helps maintain the solution distribution. Otherwise, it will be discarded during environmental selection. In this sense, elite refinement is used as a conservative local exploitation mechanism, while the final retention of refined candidates remains controlled by objective evaluation and population-level selection.

Figure 3 illustrates the main workflow of the rank-memory-guided search mechanism. Algorithm 2 gives the corresponding pseudo-code.

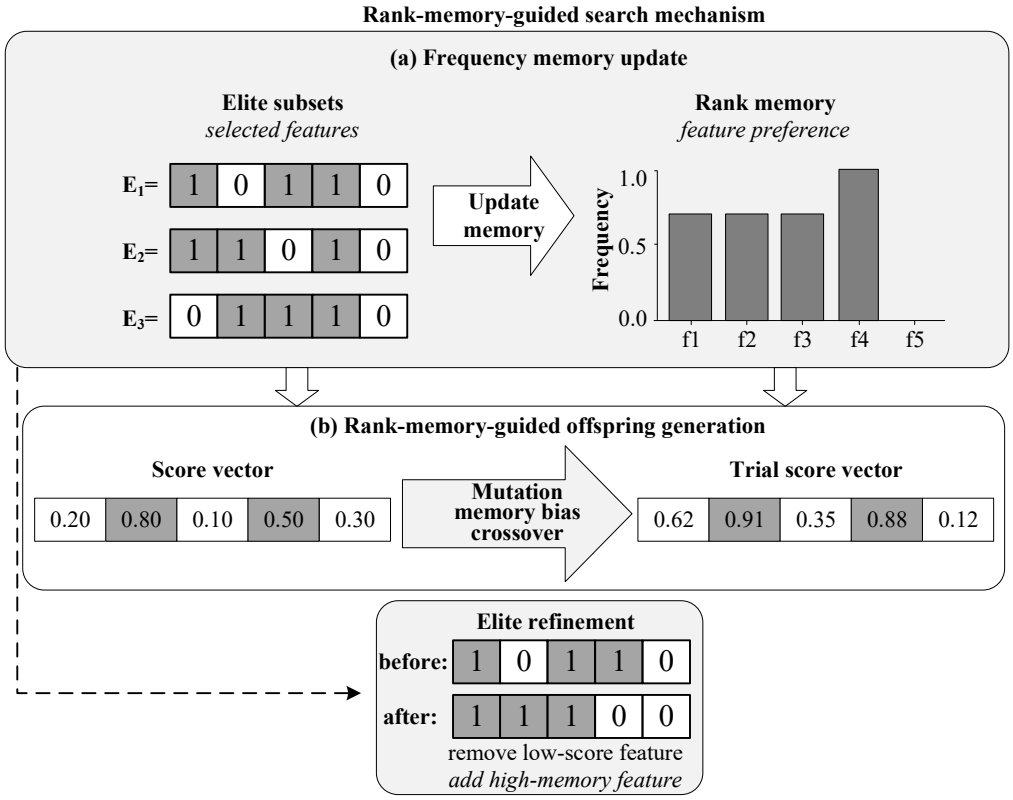


Figure 3. Rank-memory-guided search mechanism in CORAL. Gray cells highlight feature positions involved in the memory-guided search process, whereas white cells indicate inactive positions.

Algorithm 2: Rank-memory search

Input: Current population $P_t = \{\langle \mathbf{s}_i, k_i \rangle\}_{i=1}^{N}$; frequency memory $\mathbf{f}^{(t)}$
Output: Offspring population P_t^o

Select the elite set $\mathcal{E}^{(t)}$ from P_t according to non-dominated ranking and crowding distance
for $j = 1$ **to** D **do**
  Compute the current elite frequency $q_j^{(t)}$ according to Equation (10)
  Update the memory value $f_j^{(t+1)}$ according to Equation (11)
$P_t^o \leftarrow \varnothing$
for $i = 1$ **to** N **do**
  Randomly select three mutually different indices r_1, r_2, and r_3 from $\{1, 2, \ldots, N\} \backslash \{i\}$
  Generate the mutant score vector according to Equation (12)
  Generate the trial cardinality according to Equation (13)
  Apply binomial crossover according to Equation (14)
  Clip the trial score vector into $[-10, 10]$
  Set the offspring cardinality to the trial cardinality with probability 0.5; otherwise, inherit the parent cardinality
  Decode the trial score–cardinality representation into a binary feature subset using Top-K decoding
  Evaluate the decoded subset by the two objective functions
  $P_t^o \leftarrow P_t^o \cup \{\text{offspring individual}\}$
return P_t^o

3.3. Stratified Environmental Selection Strategy

In high-dimensional multi-objective feature selection, if environmental selection relies only on non-dominated sorting and crowding distance, the population may gradually concentrate around a specific feature-subset size, resulting in insufficient solution distribution across different sparsity regions. To alleviate this issue, CORAL introduces a feature-number-based stratified environmental selection strategy, which maintains solution-set diversity across different subset-size regions while preserving convergence pressure.

As shown in Figure 4, candidate solutions are first divided into B sparsity layers according to the number of selected features. Then, based on non-dominated sorting and crowding-distance sorting, the algorithm retains candidate solutions sequentially under layer-wise capacity constraints. Let $\mathcal{C}_t$ denote the candidate population obtained by merging the parent population, offspring population, and locally refined solutions. The bth sparsity layer is denoted as $\mathcal{I}_b = [e_b, e_{b+1})$, where $b = 1, 2, \ldots, B$, and the boundaries $\{e_b\}_{b=1}^{B+1}$ are obtained by equally partitioning the allowable feature-number range. To prevent candidate solutions from being over-concentrated in a few layers, the maximum number of retained solutions in each layer is set to $\lceil N/B \rceil$. If the next-generation population size is still smaller than N after stratified filtering, the remaining slots are filled from the residual candidates according to the global sorted order.

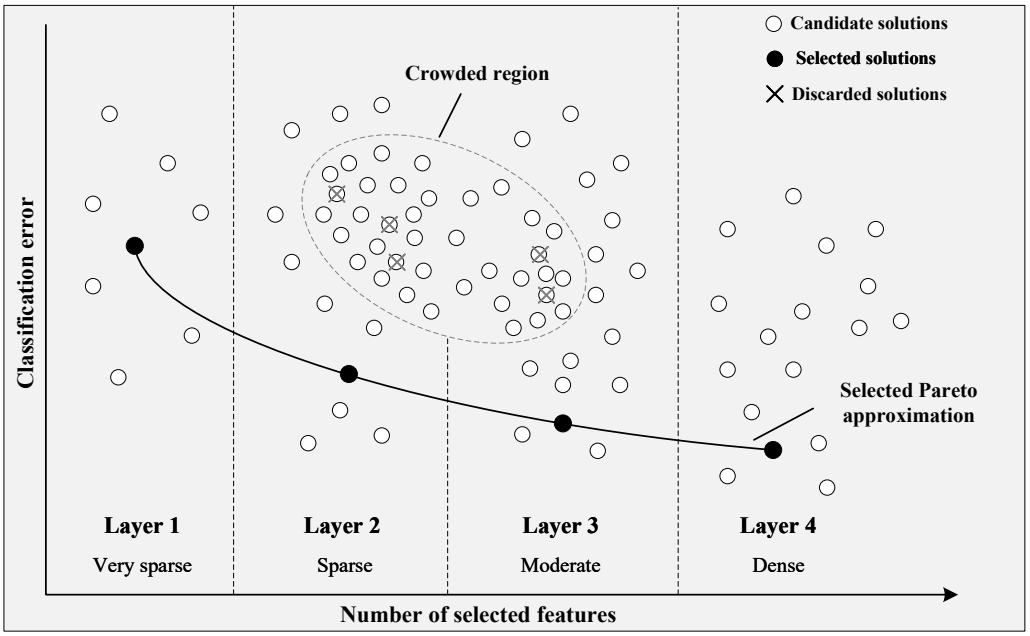


Figure 4. Stratified environmental selection strategy for maintaining sparsity-aware solution distribution across different feature-number layers.

The difference between this strategy and conventional environmental selection is that it considers not only the non-dominated relationship and crowding degree in the objective space but also the distribution of solutions along the feature-number dimension. In high-dimensional feature selection, many high-ranking candidates may have similar feature numbers. If they are selected only according to Pareto rank and crowding distance, the next population may concentrate in a narrow sparsity region. The layer-wise capacity constraint limits the number of retained solutions in each feature-number interval, so that competitive solutions from different sparsity levels still have the opportunity to survive. This helps maintain the feature-number coverage of the final Pareto solution set. Algorithm 3 presents the main procedure of stratified environmental selection.

Algorithm 3: Stratified environmental selection

Input: Candidate population $\mathcal{C}_t$; target population size N; number of sparsity layers B

Output: Next-generation population P_{t+1}

Compute the non-dominated rank and crowding distance of $\mathcal{C}_t$
Sort all candidates by ascending rank and descending crowding distance
Partition candidates into B sparsity layers according to feature cardinality
$P_{t+1} \leftarrow \varnothing$
foreach *candidate A in the sorted order* **do**
    if *the layer of A is not full* **then**
        $P_{t+1} \leftarrow P_{t+1} \cup \{A\}$
    if $|P_{t+1}| = N$ **then**
        break
if $|P_{t+1}| < N$ **then**
    Fill the remaining slots using the residual candidates in the same sorted order
return P_{t+1}

3.4. Computational Complexity Analysis

The main computational cost of CORAL comes from Top-K decoding, rank-memory-guided search, elite local refinement, and environmental selection. Let N denote the population size, D the number of features, M the number of objectives, N_m the number of elite individuals for local refinement, and $\mathcal{E}$ the cost of one fitness evaluation. In the initialization stage, generating the continuous score matrix and performing Top-K decoding require a structural cost of $O(ND \log D)$, and the corresponding initial fitness evaluation cost is $O(N\mathcal{E})$.

In each generation, frequency-memory updating requires counting feature occurrences in elite individuals, and its cost is no more than $O(ND)$. Rank-memory-guided search generates candidate solutions in the continuous score space and performs Top-K decoding and fitness evaluation for the new individuals. Therefore, its main cost is $O(ND \log D + N\mathcal{E})$. Elite local refinement is applied only to N_m individuals, and its additional cost is approximately $O(N_m D \log D + N_m \mathcal{E})$. In the environmental selection stage, the main cost comes from non-dominated sorting and crowding-distance calculation. Since the size of the candidate population is on the same order as $2N + N_m$, the dominant complexity can be approximated as $O(MN^2)$. Layer partitioning and sequential filtering introduce only additional linear costs.

In summary, if fitness evaluation is not considered, the per-generation structural complexity of CORAL is mainly $O(ND \log D + MN^2 + N_m D \log D)$. If fitness evaluation is included, an additional cost of $O((N + N_m)\mathcal{E})$ is required in each generation. In high-dimensional feature selection tasks, fitness evaluation is usually dominated by classifier evaluation and cross-validation. Therefore, the additional costs introduced by rank memory, elite local refinement, and stratified environmental selection in CORAL are relatively controllable.

4. Experimental Setup

To comprehensively evaluate the effectiveness of CORAL in multi-objective feature selection, experiments were conducted on 18 classification datasets with different dimensionalities and sample sizes. CORAL was compared with six representative multi-objective

feature selection algorithms. For fair comparison, all algorithms used the same data partitioning strategy, preprocessing procedure, evaluation protocol, and termination condition.

4.1. Classification Datasets

Eighteen public classification datasets were selected as benchmark datasets in this study. These datasets differed substantially in feature dimensionality, sample size, and data complexity. The number of features ranged from 10 to 12,533, and the number of instances ranged from 64 to 5473, allowing the adaptability and robustness of the compared algorithms to be evaluated across feature selection tasks of different scales. According to the feature dimensionality, the datasets were divided into four categories: low-dimensional datasets ($D < 500$), medium-dimensional datasets ($500 \leq D < 2000$), high-dimensional datasets ($2000 \leq D < 10{,}000$), and ultra-high-dimensional datasets ($D \geq 10{,}000$). This categorization was used only to describe the scale differences among datasets and was not involved in algorithm training or performance evaluation.

Several biomedical datasets, such as SRBCT, DLBCL, ALLAML, Leukemia2, and Tumors11, have far more features than samples. These small-sample high-dimensional datasets are prone to redundancy, noise, and overfitting, and are therefore suitable for evaluating the robustness and dimensionality reduction ability of feature selection algorithms [10,19,28,39,40]. The basic statistics of the datasets are summarized in Table 1.

Table 1. Statistics of the datasets used in the experiments.

No.	Dataset	Features	Instances	Type
1	Page_blocks	10	5473	Low-dim
2	Hill_Valley_with_noise	100	1212	Low-dim
3	Urban_land_cover	147	117	Low-dim
4	MUSK1	166	476	Low-dim
5	Semeion_handwritten_digit	256	1593	Low-dim
6	Madelon	500	1400	Mid-dim
7	ISOLET	617	1091	Mid-dim
8	QSAR_androgen_receptor	1024	1180	Mid-dim
9	SRBCT	2308	83	High-dim
10	DbWorldEmails	4702	44	High-dim
11	DLBCL	5469	77	High-dim
12	Prostate6033	6033	102	High-dim
13	ALLAML	7129	72	High-dim
14	Arcene_8k	8000	200	High-dim
15	Carcinom	9182	174	High-dim
16	Prostate_Tumor	10,509	102	Ultra-high
17	Leukemia2	11,225	72	Ultra-high
18	Tumors11	12,533	174	Ultra-high

During the experiments, each dataset was divided into training and test sets using a stratified 80%/20% split. This split was repeated across 30 independent runs with different random seeds. For each dataset and each run, the training–test partition was generated once using a predefined random seed and then shared by all compared algorithms, ensuring that all algorithms were evaluated under the same training–test partition and cross-validation setting.

The training set was used for feature-subset search and cross-validation-based fitness evaluation, whereas the test set was used only for final evaluation. To avoid data leakage, the test set was not used during evolutionary search, including fitness evaluation and population updating. Z-score standardization was performed within each cross-validation fold, where the mean and standard deviation were estimated from the training fold and

then applied to the validation fold. For final test evaluation, the standardization parameters were estimated from the full training set and then applied to the held-out test set.

We acknowledge that small-sample high-dimensional datasets, such as SRBCT, DbWorldEmails, DLBCL, ALLAML, and Leukemia2, are sensitive to data partitioning. Therefore, the reported results should be interpreted as repeated hold-out evaluations under a controlled and fair comparison protocol rather than as estimates from a fully nested resampling procedure.

4.2. Comparison Algorithms

To verify the effectiveness of CORAL, six representative multi-objective feature selection algorithms were selected as comparison methods, including NSGA-II, MOEA/D, MOEA-NZD, HEA, MFFS, and PRDH. These algorithms cover classical general-purpose MOEA frameworks, large-scale sparse multi-objective optimization methods, and representative algorithms designed for feature selection tasks in recent years.

NSGA-II is a classical Pareto-dominance-based algorithm. Its fast non-dominated sorting and crowding-distance mechanisms can maintain convergence pressure while preserving diversity in the objective space [11]. In multi-objective feature selection, NSGA-II has been used to simultaneously minimize classification error and the number of selected features, thereby obtaining a set of Pareto trade-off feature subsets [32]. MOEA/D is a classical decomposition-based multi-objective evolutionary algorithm. It decomposes a multi-objective optimization problem into a set of scalar subproblems using weight vectors and performs evolutionary search through neighborhood collaboration [12]. Therefore, NSGA-II and MOEA/D represent two general multi-objective optimization paradigms based on Pareto dominance and decomposition, respectively.

In addition to these classical algorithms, MOEA-NZD, HEA, MFFS, and PRDH were also selected for further comparison. MOEA-NZD is designed for large-scale sparse multi-objective optimization and enhances sparse search by identifying nonzero decision variables, making it suitable for evaluating algorithmic competitiveness in high-dimensional sparse feature spaces [41]. HEA is a general evolutionary optimization algorithm based on hyper-dominance degree and provides a baseline from the perspective of alternative selection pressure [42]. MFFS is a multiform multi-objective feature selection algorithm that constructs auxiliary single-objective feature selection tasks and exchanges information with the original multi-objective task to improve search efficiency and solution-set distribution [33]. PRDH emphasizes classification-performance preference through problem reformulation and improves population diversity and search stability in multi-objective feature selection by handling duplicate solutions [43]. These algorithms form comprehensive baselines from different perspectives, including general multi-objective evolution, decomposition-based search, sparse optimization, multiform auxiliary search, hyper-dominance selection pressure, and problem reformulation with duplication handling.

All comparison algorithms were implemented under the same multi-objective feature selection formulation, in which the classification error rate and the selected-feature ratio were minimized simultaneously. Except for their own search mechanisms, all algorithms used the same data partitioning, classifier setting, maximum number of fitness evaluations, and number of independent runs. Therefore, the comparison results mainly reflect the differences in search capability among the algorithms. The parameters of the comparison algorithms were set according to the original literature or the recommended settings in the PlatEMO platform.

4.3. Performance Indicators

Five performance indicators were used to evaluate the algorithms, including hypervolume (HV), inverted generational distance (IGD), Macro-F_1 score, average selected feature number (AFN), and running time. HV and IGD were used to evaluate the convergence and distribution quality of the obtained non-dominated solution sets. Macro-F_1 score was used to measure the classification performance of the selected feature subsets on the test set. AFN was used to reflect the compactness of feature subsets, and running time was used to evaluate computational efficiency.

HV is a widely used solution-set quality indicator in evolutionary multi-objective optimization. It reflects both convergence and distribution by measuring the volume dominated by the obtained solution set in the objective space [44,45]. In this study, HV was used to measure the dominated region of the non-dominated solution set obtained by each algorithm. A larger HV value indicates better solution-set quality. Before calculating HV, the objective values were normalized to the interval $[0, 1]$, and the reference point was set to $(1, 1)$.

IGD is a commonly used distance-based performance indicator in multi-objective optimization. It measures the approximation quality of the obtained solution set with respect to a reference front and can reflect convergence and diversity to some extent [46,47]. Since the true Pareto front is usually unknown in feature selection problems, this study merged the non-dominated solutions obtained by all algorithms across all independent runs and extracted the non-dominated solutions from them as an approximate reference front.

Macro-F_1 score was used to evaluate the external classification performance of the final non-dominated solution set on the test set, especially for multi-class or class-imbalanced datasets. It should be noted that Macro-F_1 score was not involved in the fitness calculation during the evolutionary process but was used only as an external evaluation indicator in the test stage. For the final non-dominated solution set obtained in each independent run, all candidate solutions were re-evaluated on the test set, and the highest Macro-F_1 score was recorded. Therefore, this indicator should be interpreted as the best external test performance found within the final non-dominated solution set, rather than as the performance of a pre-selected deployment model.

AFN denotes the number of features contained in the representative solution corresponding to the highest test Macro-F_1 score. Specifically, for each independent run, the solution with the highest test Macro-F_1 score was first identified from the final non-dominated solution set, and the number of selected features in this solution was recorded. The feature numbers were then averaged over 30 independent runs. Therefore, AFN should be interpreted jointly with Macro-F_1 score to evaluate the trade-off between external classification performance and feature compression ability. It should not be understood as the average number of features over the entire non-dominated solution set. Accordingly, AFN is reported only as an external descriptive indicator associated with the test-best solution and was not used during evolutionary search. Running time recorded the average execution time of each algorithm on each dataset and was used to measure computational efficiency and practical applicability.

To examine whether the performance differences between algorithms were statistically significant, the Wilcoxon signed-rank test was used to perform pairwise comparisons between CORAL and each comparison algorithm, with the significance level set to $\alpha = 0.05$. The symbols "+", "−", and "=" indicate that the corresponding comparison algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively. It should be noted that pairwise tests are suitable for comparing two algorithms, whereas overall comparison among multiple algorithms across multiple datasets usually requires rank-based nonparametric multiple-comparison methods [48,49].

The Friedman test was used to compare the average rankings of all algorithms across datasets, and the corresponding statistic and p-value are reported to assess overall ranking differences. When significant overall differences were observed, a Holm post hoc comparison was further conducted using CORAL as the control algorithm.

4.4. Parameter Settings

All experiments were implemented in MATLAB R2023b (MathWorks, Natick, MA, USA) based on the PlatEMO v4.13 platform [50]. Each algorithm was independently run 30 times on each dataset. For each dataset and each independent run, the training–test partition was generated once using a predefined random seed and then shared by all compared algorithms. This ensured that all algorithms were evaluated under the same data partition and cross-validation protocol. The population size was set to $N = 100$, and the maximum number of fitness evaluations was set to MaxFES = 10,000.

A weighted K-nearest neighbor (weighted KNN) classifier was used as the wrapper evaluator during evolutionary search. Owing to its simple structure and few parameters, KNN is commonly used to evaluate high-dimensional classification performance after feature selection and can be embedded into wrapper frameworks to assess the quality of candidate feature subsets [28,39]. Unlike conventional KNN with equal voting weights, weighted KNN adjusts the contribution of neighbors according to sample distances, thereby reflecting the different influences of neighboring samples on classification decisions to some extent [38].

The use of weighted KNN as the search-time evaluator was intended to keep the wrapper evaluation protocol consistent across all algorithms and datasets. To examine whether the selected feature subsets remained effective under a classifier different from the one used during search, an additional external evaluation using a linear support vector machine (SVM) was conducted on representative datasets, as reported in Section 5.5.

To balance computational efficiency and classification stability, the number of neighbors was fixed at $K_{\mathrm{NN}} = 5$ for all algorithms and datasets. During evolution, the classification error rate of each candidate feature subset was calculated by five-fold cross-validation on the training set. For very small class sizes, the number of folds was adjusted to $\min(5, n_{\min})$, where $n_{\min}$ denotes the number of samples in the smallest class of the training set. The test set was not involved in fitness evaluation.

The parameters of the comparison algorithms were set according to the original literature or the recommended settings in the PlatEMO platform.

For CORAL, the parameters considered in the sensitivity analysis were the number of sparsity layers B, the number of elite individuals for local refinement N_m, the memory update coefficient ρ, and the memory-guidance strength γ. Here, ρ controls the update rate between historical memory and current elite frequency, while γ controls the strength of the memory-guided bias in score-space mutation. Based on the sensitivity results, CORAL was configured with $B = 5$, $N_m = 3$, $\rho = 0.4$, and $\gamma = 0.4$ in the comparative experiments.

The remaining operator-level constants were fixed for reproducibility and were not treated as the primary tuning parameters. Specifically, the differential scaling factor in score-space mutation was set to $F = 0.5$, the score crossover probability to $p_c = 0.9$, the cardinality scaling factor to $\lambda_k = 0.3$, and the Gaussian perturbation strengths for the score vector and cardinality variable to $\sigma = 0.1$ and $\sigma_k = 0.5$, respectively. The allowable cardinality range was set to $K_{\min} = 1$ and $K_{\max} = \mathrm{round}(0.2D)$.

In summary, all compared algorithms were evaluated under the same data partitions, preprocessing procedure, classifier setting, maximum number of fitness evaluations, and independent-run protocol. Therefore, the reported differences mainly reflect the search

behavior of the algorithms rather than differences in the experimental protocol. The parameter settings of the compared algorithms are summarized in Table 2.

Table 2. Parameter settings of the compared algorithms.

Algorithm	Parameter Settings
NSGA-II	$p_c = 1.0$, $p_m = 1/D$, $\eta_c = 20$, $\eta_m = 20$
MOEA/D	$p_c = 1.0$, $p_m = 1/D$, $T = 20$
MOEA-NZD	$p_c = 1.0$, $p_m = 1/D$
HEA	$p_c = 0.9$, $p_m = 0.1$, $\gamma = 0.5$
MFFS	$R = 3$, $\beta = 0.5$
PRDH	$p_c = 0.9$, $F = 0.5$
CORAL	$B = 5$, $N_m = 3$, $\rho = 0.4$, $\gamma = 0.4$

5. Results and Discussion

5.1. Overall Performance Analysis

Tables 3–6 summarize the results of HV, IGD, Macro-F_1, and AFN. HV and IGD are reported as mean and standard deviation over 30 independent runs, while Macro-F_1 and AFN are reported as average values. The best-result formatting and statistical-symbol definitions are provided in the corresponding table captions.

Table 3 reports the HV comparison results. CORAL obtains favorable HV values on most datasets, suggesting that its final non-dominated sets generally provide good objective-space coverage under the given evaluation budget. Compared with classical multi-objective evolutionary algorithms such as HEA, MOEA/D, and NSGA-II, the advantage of CORAL is more evident on high-dimensional and ultra-high-dimensional datasets. This result suggests that binary evolutionary search may be more affected by redundancy and search-space expansion in large-scale feature spaces. In contrast, CORAL can more effectively focus the search on potentially promising feature subspaces through the joint continuous score–cardinality representation and the rank-memory-guided search mechanism. This observation is consistent with previous studies on high-dimensional feature selection, which have shown that redundant and irrelevant features, together with search-space expansion, can significantly affect search efficiency and solution-set quality [5,10,19].

The IGD comparison in Table 4 further reflects the ability of the obtained solution sets to approximate the reference front. Compared with HV, the advantages of different algorithms on IGD are more scattered, indicating that different search strategies exhibit different behaviors in terms of front approximation and solution-set coverage. CORAL obtains the best or competitive IGD values on several datasets and maintains stable performance on most high-dimensional datasets. CORAL therefore provides a relatively balanced trade-off between objective-space coverage and approximation to the reference front. Meanwhile, PRDH and MOEA-NZD obtain lower IGD values on some datasets, suggesting that problem reformulation, duplicate-solution handling, and sparse-variable identification can have positive effects on IGD performance.

For the external classification and compactness indicators, Table 5 reports the Macro-F_1 results, whereas Table 6 presents the AFN results. CORAL achieves high or competitive Macro-F_1 scores on most datasets, indicating that its final non-dominated solution set usually contains feature subsets with competitive test classification performance. In terms of AFN, PRDH obtains smaller feature subset sizes on several high-dimensional datasets, reflecting its strong tendency toward feature compression. In contrast, CORAL does not simply pursue the smallest number of selected features on all datasets. Instead, it aims to obtain relatively compact feature subsets while maintaining high Macro-F_1 performance.

Therefore, the Macro-F_1 and AFN results should be interpreted jointly: CORAL places more emphasis on balancing classification performance and feature compression rather than merely minimizing the number of selected features.

Table 3. Hypervolume (HV) comparison results of CORAL and six competing algorithms on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The symbols $+$, $-$, and $=$ indicate that the corresponding algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
1	8.7928×10^{-1} (1.14×10^{-16}) +	8.8038×10^{-1} (3.42×10^{-16}) +	8.8019×10^{-1} (8.78×10^{-4}) +	8.7102×10^{-1} (1.14×10^{-16}) =	**8.8865×10^{-1}** (2.28×10^{-16}) +	8.8038×10^{-1} (3.42×10^{-16}) +	8.7102×10^{-1} (1.14×10^{-16})
2	6.4725×10^{-1} (1.85×10^{-2}) −	6.7683×10^{-1} (9.13×10^{-3}) −	6.7271×10^{-1} (1.35×10^{-2}) =	6.5470×10^{-1} (1.21×10^{-2}) −	6.7649×10^{-1} (1.03×10^{-2}) −	6.1912×10^{-1} (2.43×10^{-2}) −	**6.8991×10^{-1}** (9.44×10^{-3})
3	9.0292×10^{-1} (4.05×10^{-2}) −	9.5423×10^{-1} (1.14×10^{-2}) −	9.5134×10^{-1} (2.77×10^{-2}) =	8.9974×10^{-1} (1.89×10^{-2}) −	9.5064×10^{-1} (1.10×10^{-3}) −	9.3677×10^{-1} (8.94×10^{-2}) −	**9.8795×10^{-1}** (8.70×10^{-3})
4	9.2867×10^{-1} (2.00×10^{-2}) −	9.8250×10^{-1} (7.49×10^{-3}) −	9.7586×10^{-1} (8.73×10^{-3}) =	9.1177×10^{-1} (1.34×10^{-2}) −	9.7860×10^{-1} (7.49×10^{-3}) =	9.7598×10^{-1} (8.92×10^{-3}) =	**9.8897×10^{-1}** (7.28×10^{-3})
5	9.5293×10^{-1} (2.27×10^{-2}) −	**9.9659×10^{-1}** (5.47×10^{-5}) +	9.9595×10^{-1} (8.14×10^{-4}) =	9.4002×10^{-1} (1.68×10^{-2}) −	9.9520×10^{-1} (2.21×10^{-3}) =	8.6468×10^{-1} (4.58×10^{-2}) −	9.9653×10^{-1} (1.83×10^{-4})
6	5.9113×10^{-1} (2.77×10^{-2}) −	**9.1849×10^{-1}** (7.35×10^{-3}) +	8.9070×10^{-1} (4.66×10^{-2}) =	5.0926×10^{-1} (1.86×10^{-2}) −	7.9117×10^{-1} (2.59×10^{-2}) −	8.9253×10^{-1} (9.59×10^{-2}) −	9.0547×10^{-1} (2.02×10^{-2})
7	7.3678×10^{-1} (1.19×10^{-2}) −	9.3756×10^{-1} (1.64×10^{-2}) −	9.1651×10^{-1} (1.19×10^{-2}) −	6.9494×10^{-1} (1.32×10^{-2}) −	8.9371×10^{-1} (1.04×10^{-2}) −	9.2766×10^{-1} (8.44×10^{-3}) =	**9.4331×10^{-1}** (7.01×10^{-3})
8	7.1183×10^{-1} (1.13×10^{-2}) −	9.5003×10^{-1} (6.95×10^{-3}) −	9.4928×10^{-1} (4.76×10^{-3}) −	7.2287×10^{-1} (1.11×10^{-2}) −	8.7125×10^{-1} (8.96×10^{-3}) −	9.1429×10^{-1} (1.56×10^{-2}) −	**9.5203×10^{-1}** (4.28×10^{-3})
9	6.6761×10^{-1} (1.47×10^{-2}) −	**9.9951×10^{-1}** (2.86×10^{-5}) +	9.9876×10^{-1} (3.87×10^{-4}) =	6.9987×10^{-1} (2.66×10^{-2}) −	8.6776×10^{-1} (8.48×10^{-3}) −	7.4970×10^{-1} (8.91×10^{-2}) −	9.9925×10^{-1} (3.05×10^{-4})
10	4.5144×10^{-1} (5.53×10^{-2}) −	9.9915×10^{-1} (2.94×10^{-3}) =	9.9954×10^{-1} (1.58×10^{-4}) −	4.8251×10^{-1} (6.97×10^{-2}) −	7.5539×10^{-1} (2.67×10^{-2}) −	8.2370×10^{-1} (1.35×10^{-1}) −	**9.9981×10^{-1}** (1.14×10^{-4})
11	6.0420×10^{-1} (2.22×10^{-3}) −	9.9978×10^{-1} (2.23×10^{-4}) =	9.9974×10^{-1} (1.00×10^{-4}) −	6.4583×10^{-1} (4.11×10^{-3}) −	7.7653×10^{-1} (7.02×10^{-3}) −	9.2712×10^{-1} (3.73×10^{-2}) −	**9.9983×10^{-1}** (5.59×10^{-5})
12	5.4914×10^{-1} (4.44×10^{-3}) −	9.6984×10^{-1} (2.10×10^{-2}) −	9.8028×10^{-1} (1.96×10^{-2}) −	5.7854×10^{-1} (3.69×10^{-3}) −	6.3610×10^{-1} (4.18×10^{-2}) −	9.2951×10^{-1} (6.59×10^{-2}) −	**9.9984×10^{-1}** (1.89×10^{-4})
13	5.9887×10^{-1} (5.49×10^{-3}) −	9.9973×10^{-1} (4.61×10^{-4}) −	9.9967×10^{-1} (1.12×10^{-4}) =	6.1440×10^{-1} (1.12×10^{-2}) −	7.4498×10^{-1} (6.25×10^{-3}) −	9.0897×10^{-1} (6.11×10^{-2}) −	**9.9987×10^{-1}** (1.12×10^{-4})
14	5.5263×10^{-1} (8.00×10^{-3}) −	9.9456×10^{-1} (1.02×10^{-2}) −	9.9605×10^{-1} (8.34×10^{-3}) −	5.5915×10^{-1} (6.89×10^{-3}) −	6.8933×10^{-1} (1.02×10^{-2}) −	9.7257×10^{-1} (8.42×10^{-2}) −	**9.9863×10^{-1}** (1.04×10^{-2})
15	5.3689×10^{-1} (7.88×10^{-3}) −	**9.7917×10^{-1}** (1.38×10^{-2}) +	9.7482×10^{-1} (1.47×10^{-2}) +	5.5110×10^{-1} (7.32×10^{-3}) −	6.6859×10^{-1} (1.19×10^{-2}) −	9.7838×10^{-1} (1.10×10^{-2}) +	9.7210×10^{-1} (1.98×10^{-4})
16	4.5689×10^{-1} (1.00×10^{-2}) −	9.6508×10^{-1} (4.32×10^{-2}) +	**9.8362×10^{-1}** (2.22×10^{-2}) +	4.6734×10^{-1} (7.17×10^{-3}) −	5.7355×10^{-1} (1.05×10^{-2}) −	8.1129×10^{-1} (4.49×10^{-2}) −	9.5403×10^{-1} (2.01×10^{-2})
17	5.4587×10^{-1} (3.05×10^{-3}) −	9.9838×10^{-1} (2.79×10^{-3}) =	9.9971×10^{-1} (9.71×10^{-5}) =	5.5740×10^{-1} (3.64×10^{-3}) −	6.6367×10^{-1} (4.04×10^{-3}) −	7.7916×10^{-1} (4.90×10^{-2}) −	**9.9992×10^{-1}** (1.66×10^{-4})
18	5.0455×10^{-1} (2.31×10^{-3}) −	9.6445×10^{-1} (1.32×10^{-2}) −	9.6535×10^{-1} (1.69×10^{-2}) −	5.1146×10^{-1} (3.94×10^{-3}) −	6.1811×10^{-1} (1.23×10^{-2}) −	9.6504×10^{-1} (2.14×10^{-2}) =	**9.7154×10^{-1}** (1.24×10^{-2})
+/−/=	1/17/0	6/9/3	3/7/8	0/17/1	1/15/2	2/13/3	–

Table 4. Inverted generational distance (IGD) comparison results of CORAL and six competing algorithms on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The symbols $+$, $-$, and $=$ indicate that the corresponding algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
1	6.02×10^{-2} (7.1×10^{-18}) +	1.74×10^{-1} (1.4×10^{-3}) +	1.85×10^{-1} (9.8×10^{-4}) +	2.28×10^{-1} (0.0×10^{0}) +	**0.00×10^{0}** (0.0×10^{0}) +	1.85×10^{-1} (0.0×10^{0}) +	2.45×10^{-1} (0.0×10^{0})
2	1.09×10^{-1} (1.5×10^{-2}) −	9.35×10^{-2} (3.9×10^{-3}) =	9.67×10^{-2} (5.8×10^{-3}) −	1.03×10^{-1} (5.7×10^{-3}) −	**2.73×10^{-2}** (2.1×10^{-2}) +	1.15×10^{-1} (1.1×10^{-2}) −	7.46×10^{-2} (6.7×10^{-3})
3	2.19×10^{-1} (7.1×10^{-2}) −	9.65×10^{-2} (7.8×10^{-3}) +	1.10×10^{-1} (1.9×10^{-2}) =	2.52×10^{-1} (2.1×10^{-2}) −	**2.19×10^{-2}** (3.0×10^{-2}) +	1.12×10^{-1} (3.9×10^{-2}) −	1.01×10^{-1} (4.4×10^{-3})

Table 4. *Cont.*

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
4	1.60×10^{-1} (2.0×10^{-2}) −	8.98×10^{-2} (4.1×10^{-3}) −	9.31×10^{-2} (2.9×10^{-3}) −	1.79×10^{-1} (8.9×10^{-3}) −	**6.99×10^{-2}** (4.3×10^{-2}) =	8.70×10^{-2} (2.3×10^{-3}) +	8.89×10^{-2} (3.6×10^{-3})
5	4.05×10^{-1} (1.2×10^{-2}) −	2.68×10^{-1} (1.3×10^{-2}) −	2.78×10^{-1} (2.6×10^{-2}) −	3.94×10^{-1} (1.4×10^{-2}) −	**1.78×10^{-1}** (1.4×10^{-1}) =	1.87×10^{-1} (3.3×10^{-2}) +	1.97×10^{-1} (2.0×10^{-2})
6	3.38×10^{-1} (2.6×10^{-2}) −	7.88×10^{-2} (3.0×10^{-3}) −	9.17×10^{-2} (2.8×10^{-2}) =	4.40×10^{-1} (1.7×10^{-2}) −	1.78×10^{-1} (1.3×10^{-2}) −	9.12×10^{-2} (6.1×10^{-2}) −	**7.60×10^{-2}** (9.9×10^{-3})
7	2.76×10^{-1} (1.5×10^{-2}) −	3.22×10^{-2} (4.9×10^{-3}) −	3.48×10^{-2} (4.7×10^{-3}) −	3.17×10^{-1} (1.3×10^{-2}) −	1.50×10^{-1} (1.5×10^{-2}) −	2.54×10^{-2} (9.6×10^{-3}) −	**2.45×10^{-2}** (3.5×10^{-3})
8	3.04×10^{-1} (1.2×10^{-2}) −	9.20×10^{-2} (3.7×10^{-3}) −	**8.70×10^{-2}** (1.9×10^{-3}) +	2.95×10^{-1} (1.3×10^{-2}) −	1.50×10^{-1} (7.4×10^{-3}) −	1.07×10^{-1} (9.6×10^{-3}) −	8.73×10^{-2} (2.2×10^{-3})
9	6.08×10^{-1} (8.2×10^{-3}) −	2.71×10^{-1} (3.4×10^{-2}) −	1.85×10^{-1} (6.0×10^{-2}) +	5.73×10^{-1} (2.0×10^{-2}) −	4.09×10^{-1} (1.9×10^{-2}) −	**1.64×10^{-1}** (5.8×10^{-2}) +	1.89×10^{-1} (2.0×10^{-2})
10	6.70×10^{-1} (1.6×10^{-2}) −	3.75×10^{-1} (6.9×10^{-2}) =	3.38×10^{-1} (1.1×10^{-1}) =	6.55×10^{-1} (2.3×10^{-2}) −	5.20×10^{-1} (5.4×10^{-2}) −	3.63×10^{-1} (1.0×10^{-1}) −	**2.57×10^{-1}** (1.2×10^{-3})
11	7.63×10^{-1} (1.7×10^{-3}) −	4.67×10^{-1} (6.4×10^{-2}) −	4.40×10^{-1} (7.0×10^{-2}) =	7.31×10^{-1} (3.1×10^{-3}) −	6.12×10^{-1} (1.5×10^{-2}) −	**2.43×10^{-1}** (2.4×10^{-2}) +	4.09×10^{-1} (9.6×10^{-3})
12	5.96×10^{-1} (4.3×10^{-3}) −	3.41×10^{-1} (9.8×10^{-2}) −	1.54×10^{-1} (3.2×10^{-2}) +	5.68×10^{-1} (3.5×10^{-3}) −	4.79×10^{-1} (2.6×10^{-2}) −	**9.34×10^{-2}** (2.3×10^{-2}) +	2.07×10^{-1} (1.5×10^{-2})
13	7.66×10^{-1} (5.4×10^{-3}) −	4.29×10^{-1} (4.7×10^{-2}) −	3.95×10^{-1} (9.9×10^{-2}) =	7.49×10^{-1} (1.1×10^{-2}) −	6.25×10^{-1} (2.1×10^{-2}) −	**1.84×10^{-1}** (4.8×10^{-2}) +	3.64×10^{-1} (1.8×10^{-2})
14	5.93×10^{-1} (2.9×10^{-3}) −	2.68×10^{-1} (3.1×10^{-2}) −	**1.80×10^{-1}** (2.5×10^{-2}) +	5.83×10^{-1} (6.7×10^{-3}) −	4.64×10^{-1} (8.3×10^{-3}) −	1.96×10^{-1} (1.8×10^{-2}) −	1.88×10^{-1} (5.6×10^{-3})
15	5.68×10^{-1} (3.9×10^{-3}) −	2.50×10^{-1} (6.7×10^{-2}) −	1.12×10^{-1} (1.2×10^{-2}) −	5.48×10^{-1} (9.3×10^{-3}) −	4.43×10^{-1} (1.1×10^{-2}) −	**5.56×10^{-2}** (5.0×10^{-3}) +	6.10×10^{-2} (1.0×10^{-2})
16	4.93×10^{-1} (6.7×10^{-3}) −	5.62×10^{-2} (2.3×10^{-2}) −	3.63×10^{-2} (1.6×10^{-2}) −	4.77×10^{-1} (5.0×10^{-3}) −	3.60×10^{-1} (1.3×10^{-2}) −	1.41×10^{-1} (4.9×10^{-2}) −	**1.90×10^{-2}** (1.4×10^{-2})
17	4.63×10^{-1} (3.6×10^{-3}) −	1.05×10^{-2} (3.0×10^{-3}) −	4.77×10^{-4} (2.1×10^{-4}) −	4.50×10^{-1} (4.2×10^{-3}) −	3.27×10^{-1} (4.6×10^{-3}) −	2.43×10^{-1} (5.4×10^{-2}) −	**0.00×10^{0}** (2.7×10^{-3})
18	4.83×10^{-1} (2.8×10^{-3}) −	6.86×10^{-2} (1.7×10^{-2}) −	6.41×10^{-2} (9.1×10^{-3}) −	4.76×10^{-1} (7.2×10^{-3}) −	3.68×10^{-1} (1.0×10^{-2}) −	**7.53×10^{-3}** (4.9×10^{-3}) +	1.67×10^{-2} (5.0×10^{-3})
+/−/=	1/17/0	2/14/2	5/8/5	1/17/0	3/13/2	9/9/0	–

Table 5. Comparison of macro-averaged F_1 (Macro-F_1) scores on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The +/−/= row summarizes the number of datasets on which each algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
1	0.9580	**0.9726**	0.9724	0.9580	**0.9726**	**0.9726**	0.9580
2	0.6424	0.6533	0.6500	0.6417	0.6525	0.5876	**0.6694**
3	0.9391	0.9609	0.9609	0.9565	0.9565	0.9413	**1.0000**
4	0.9884	0.9921	0.9853	0.9842	0.9916	0.9837	**1.0000**
5	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	0.8540	**1.0000**
6	0.7432	**0.9137**	0.8834	0.7448	0.8298	0.8852	0.9000
7	0.9257	0.9447	0.9204	0.9236	**0.9661**	0.9300	0.9541
8	0.9263	0.9466	0.9456	0.9307	0.9477	0.9068	**0.9492**
9	0.9938	**1.0000**	**1.0000**	0.9875	**1.0000**	0.7250	**1.0000**
10	0.7063	**1.0000**	**1.0000**	0.7250	0.9812	0.8063	**1.0000**
11	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	0.9200	**1.0000**
12	0.9000	0.9691	0.9794	0.9000	0.8884	0.9226	**1.0000**
13	**1.0000**	**1.0000**	**1.0000**	0.9929	**1.0000**	0.9000	**1.0000**
14	0.9200	0.9975	0.9962	0.9150	0.9487	0.9700	**1.0000**
15	0.9059	**0.9794**	0.9735	0.8985	0.9309	0.9765	0.9706
16	0.7600	0.9650	**0.9825**	0.7550	0.7975	0.7925	0.9500
17	0.9286	**1.0000**	**1.0000**	0.9286	0.9286	0.7571	**1.0000**
18	0.8529	0.9632	0.9632	0.8544	0.8882	0.9618	**0.9706**
+/−/=	3/15/0	6/12/0	6/12/0	2/16/0	5/13/0	0/18/0	–

Table 6. Comparison of average selected feature number (AFN) values on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The $+/-/=$ row summarizes the number of datasets on which each algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
1	**2.0**	4.0	4.1	**2.0**	4.0	4.0	**2.0**
2	11.6	7.2	9.2	11.4	8.1	**1.9**	6.0
3	10.2	6.5	7.3	12.9	5.5	6.4	**5.0**
4	21.3	16.9	13.2	22.2	15.3	**11.3**	12.0
5	16.1	3.1	3.5	22.9	4.0	**1.2**	3.0
6	141.4	7.2	7.2	207.8	52.2	7.2	**5.0**
7	152.1	107.7	43.0	179.8	89.0	**33.5**	43.0
8	267.6	55.0	18.1	264.9	138.9	**17.0**	29.0
9	834.2	3.5	8.1	760.4	346.4	**1.1**	6.0
10	1990.0	5.7	3.8	1850.0	1220.0	**1.0**	**1.0**
11	2380.0	3.0	2.4	2140.0	1360.0	**1.0**	**1.0**
12	2630.0	47.7	26.6	2430.0	3630.0	12.3	**2.0**
13	3150.0	4.0	4.5	3010.0	2020.0	**1.0**	**1.0**
14	3560.0	57.0	16.9	3480.0	2470.0	**7.0**	165.0
15	4190.0	40.8	23.9	3980.0	2980.0	**10.2**	61.0
16	4820.0	45.4	13.4	4630.0	3520.0	**1.0**	16.0
17	5140.0	22.8	7.0	5000.0	3610.0	**1.0**	**1.0**
18	5760.0	95.0	30.9	5680.0	4430.0	**15.2**	128.0
$+/-/=$	7/10/1	4/10/4	4/11/3	8/10/0	8/9/1	2/15/1	–

Overall, CORAL provides competitive solution-set quality while maintaining reasonable feature compactness and test classification performance. Although PRDH, MOEA-NZD, and MFFS perform well on certain datasets or individual indicators, CORAL shows more balanced behavior across the main evaluation criteria. To compare the overall performance of all algorithms across the 18 datasets, the Friedman test was conducted on HV, IGD, Macro-F_1, AFN, and running time. Table 7 reports the average ranks of all algorithms, together with the Friedman statistic, the corresponding p-value, and the Iman–Davenport statistic. For HV and Macro-F_1, larger values were ranked better, whereas for IGD, AFN, and running time, smaller values were ranked better.

Table 7. Friedman average ranks and overall statistical test results of the compared algorithms across five indicators.

Algorithm	HV	IGD	Macro-F_1	AFN	Runtime
HEA	6.50	6.44	5.42	6.39	4.83
MFFS	2.14	3.44	2.36	3.67	3.67
MOEA-NZD	2.94	3.03	3.25	3.31	4.78
MOEA/D	6.19	6.17	5.67	6.00	6.72
NSGA-II	4.39	3.89	3.78	4.78	4.11
PRDH	4.08	2.75	5.28	1.61	2.50
CORAL	1.75	2.28	2.25	2.25	1.39
Overall statistical test					
Friedman χ_F^2	80.64	63.57	56.20	77.19	69.05
Friedman p-value	2.64×10^{-15}	8.45×10^{-12}	2.65×10^{-10}	1.36×10^{-14}	6.41×10^{-13}
Iman–Davenport F_{ID}	50.10	24.32	18.44	42.59	30.13
Iman–Davenport p-value	3.04×10^{-28}	1.04×10^{-17}	2.06×10^{-14}	1.19×10^{-25}	1.49×10^{-20}

The Friedman results in Table 7 show that all Friedman and Iman–Davenport p-values are below 0.05, indicating significant overall differences among the compared algorithms. CORAL achieves the lowest average ranks on HV, IGD, Macro-F_1, and running time, and the second-lowest rank on AFN. PRDH obtains the lowest AFN rank, showing a stronger tendency toward feature compression.

A Holm post hoc comparison was then performed using CORAL as the control algorithm. After Holm correction at $\alpha = 0.05$, significant differences favoring CORAL were observed on HV against HEA, MOEA/D, NSGA-II, and PRDH; on IGD against HEA and MOEA/D; on Macro-F_1 against MOEA/D, HEA, and PRDH; on AFN against HEA, MOEA/D, and NSGA-II; and on running time against MOEA/D, HEA, MOEA-NZD, NSGA-II, and MFFS. The remaining pairwise comparisons were not significant after Holm correction. These results indicate that CORAL provides competitive overall performance while remaining comparable with several recent feature-selection-oriented methods on specific indicators.

5.2. Analysis of Non-Dominated Front Distribution and Convergence Behavior

To further examine the trade-off between classification performance and feature-subset compactness, Figure 5 presents the median non-dominated fronts obtained on nine representative datasets. The selected datasets cover low-dimensional tasks (Hill_Valley_with_noise, Urban_land_cover, MUSK1, and Semeion_handwritten_digit), medium-dimensional tasks (ISOLET and QSAR_androgen_receptor), and high-dimensional tasks (SRBCT, DbWorldEmails, and DLBCL). In each panel, the horizontal axis denotes the number of selected features, and the vertical axis denotes the Macro-F_1 loss, calculated as $1 - \text{Macro-}F_1$. Therefore, fronts located closer to the lower-left region indicate a more favorable balance between classification loss and feature reduction.

Across these representative cases, CORAL tends to provide competitive or lower-loss solutions while using compact feature subsets. This behavior is particularly visible on several medium- and high-dimensional datasets, where some competing algorithms retain substantially larger feature subsets to achieve comparable loss values. These results support the role of the score–cardinality representation in decoupling feature-priority modeling from subset-size control, thereby helping the search avoid unnecessarily large subsets while preserving predictive performance.

Figure 6 further reports the HV convergence curves on the same nine datasets, allowing the front-distribution results in Figure 5 to be interpreted together with the optimization dynamics. CORAL generally reaches high HV values within the early stages of evolution and maintains competitive convergence throughout the evaluation budget. In contrast, several baseline algorithms exhibit slower improvement or stabilize at lower HV values on the medium- and high-dimensional datasets. This pattern suggests that rank-memory guidance and stratified environmental selection help CORAL exploit informative feature regions while maintaining a well-distributed set of non-dominated solutions.

Taken together, the Pareto-front and HV convergence analyses indicate that CORAL does not simply optimize one aspect of the feature-selection problem. Instead, it maintains a stable trade-off among solution-set quality, feature compactness, and classification performance across representative datasets of different dimensional scales.

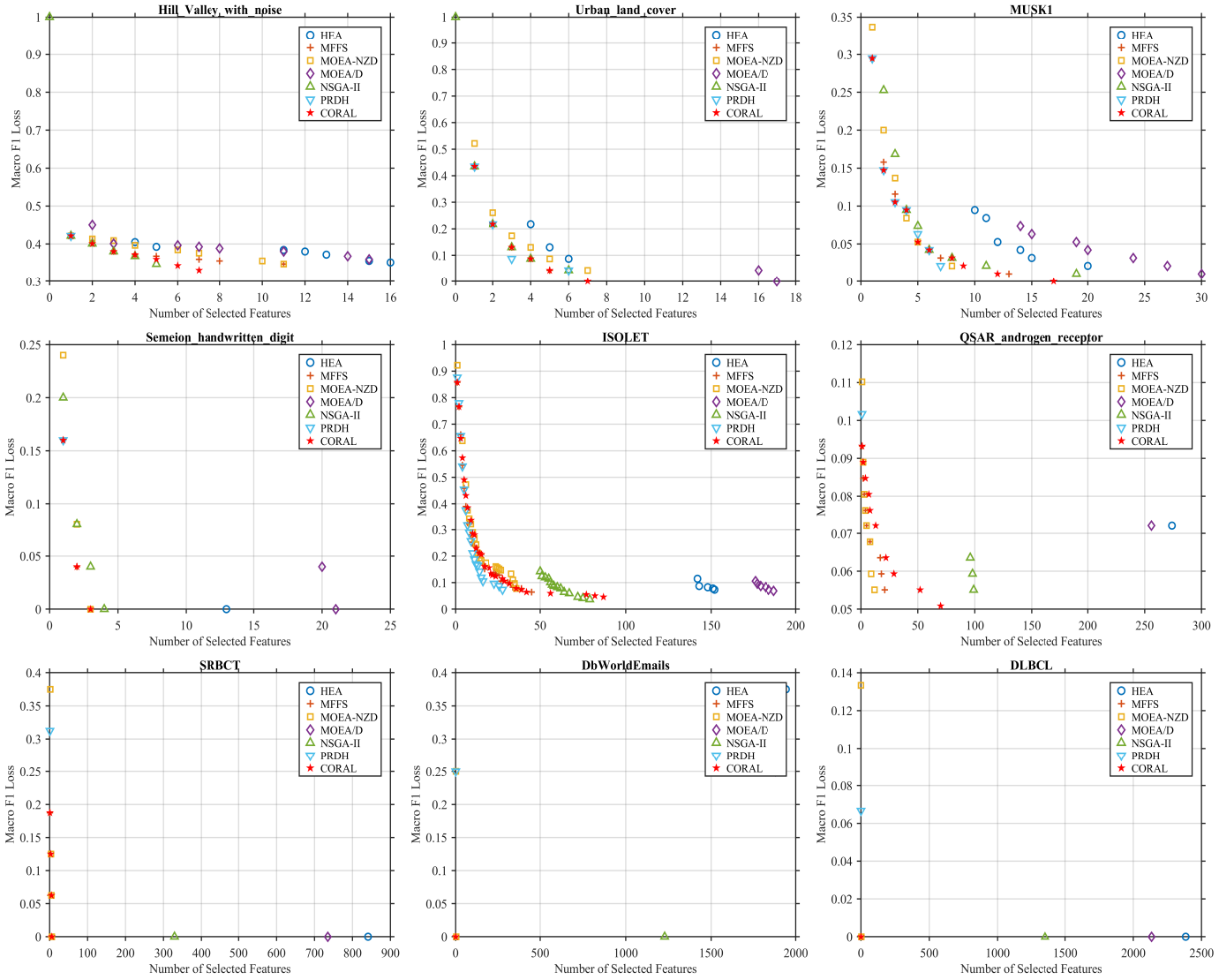


Figure 5. Median Pareto fronts obtained by different algorithms on nine representative datasets selected from the low-, medium-, and high-dimensional groups. The symbols and line styles correspond to the legends shown in the subfigures.

These visual results provide additional empirical support for the convergence and distribution analyses. The HV convergence curves reflect the optimization process under the fixed evaluation budget, while the median Pareto fronts show the final trade-offs between feature number and classification loss. The observed behavior of CORAL, together with the Friedman rankings and ablation results, supports the roles of rank-memory guidance in search direction and stratified environmental selection in sparsity-aware solution maintenance.

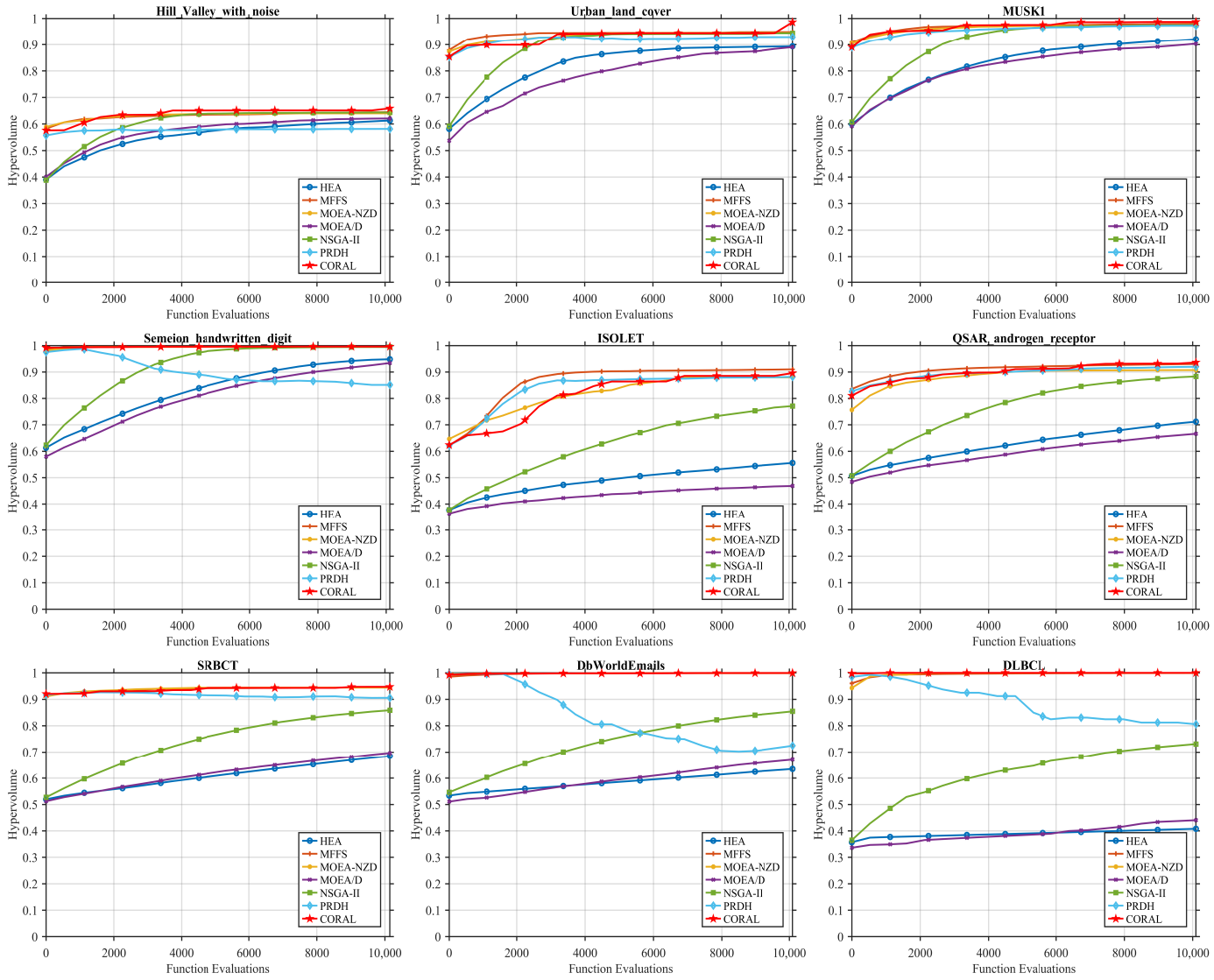


Figure 6. Hypervolume (HV) convergence curves of different algorithms on the same nine representative datasets as in Figure 5. The line styles and colors correspond to the algorithm legends shown in the subfigures.

5.3. Running Time Analysis

Running time reflects the practical scalability of feature selection algorithms in high-dimensional scenarios. The running-time comparison in Table 8 shows that CORAL achieves shorter average running time than HEA, MOEA/D, and NSGA-II on most datasets. This indicates that the additional operations introduced by rank memory, elite local refinement, and stratified environmental selection do not lead to prohibitive overhead under the fixed evaluation budget.

On the ultra-high-dimensional datasets, CORAL obtains the shortest average running time on Prostate_Tumor and Leukemia2 and remains close to the fastest method on Tumors11. Although PRDH is faster on several high-dimensional datasets, this may be related to its stronger feature-compression tendency. In contrast, CORAL aims to maintain a balance among solution-set quality, classification performance, feature compactness, and computational efficiency, rather than minimizing running time alone.

Table 8. Execution time comparison in seconds of CORAL and six competing algorithms on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The $+/-/=$ row summarizes the number of datasets on which each algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
1	146.1	164.5	162.3	153.8	152.2	184.2	**139.3**
2	100.0	89.7	97.9	110.7	87.3	91.3	**79.1**
3	66.5	69.3	71.1	72.1	63.9	69.6	**52.6**
4	74.9	76.1	82.0	84.9	75.3	77.7	**59.2**
5	67.2	69.9	72.5	74.5	67.3	69.4	**52.7**
6	406.5	164.9	198.5	618.9	285.4	**126.8**	137.8
7	322.8	147.3	197.0	445.7	240.1	**114.5**	123.6
8	885.2	176.5	278.4	968.1	567.6	**160.9**	180.8
9	84.3	83.3	90.5	109.9	79.1	75.8	**58.7**
10	89.8	103.0	94.4	119.7	86.6	85.1	**65.2**
11	141.4	117.0	124.3	196.4	114.4	93.4	**71.0**
12	181.0	203.0	320.0	265.0	1030.0	113.0	**76.9**
13	161.0	134.5	137.1	233.6	141.4	101.7	**77.0**
14	625.9	165.6	320.8	850.7	472.3	**100.9**	120.9
15	592.9	173.1	323.0	797.3	448.0	**104.5**	120.7
16	291.0	179.0	228.6	452.7	257.0	129.2	**101.6**
17	226.2	186.2	186.5	349.2	205.0	128.6	**93.9**
18	915.0	221.0	418.0	1260.0	764.0	**123.0**	150.0
$+/-/=$	0/16/2	1/15/2	0/18/0	0/18/0	0/16/2	6/10/2	–

From the perspective of scalability, the dominant cost in wrapper-based MOFS comes from repeated fitness evaluation, especially classifier evaluation and cross-validation. The structural operations in CORAL, including frequency-memory updating, score-space variation, elite local refinement, and layer-wise selection, are lightweight compared with the evaluation cost. In particular, elite local refinement is applied only to N_m elite individuals, and stratified environmental selection adds only a layer-wise filtering step after standard ranking. Therefore, the practical runtime results on datasets with more than 10,000 features suggest that CORAL remains computationally feasible for ultra-high-dimensional feature selection tasks.

5.4. Effectiveness Analysis of Key Strategies

To verify the effectiveness of the key mechanisms in CORAL, three ablated variants were constructed for comparison, namely, NoLayerSel, NoRankMem, and NoEliteRef. Specifically, NoLayerSel removes the stratified environmental selection strategy, NoRankMem removes the rank-memory-guided search mechanism, and NoEliteRef removes the elite local refinement operation. The HV comparison results of the complete CORAL framework and its ablated variants are reported in Table 9.

The overall results show that the complete CORAL framework achieves more stable HV performance on most datasets, indicating that the three key mechanisms are complementary. A quantitative summary further shows that CORAL obtains the best average HV rank (1.67) across the 18 datasets, followed by NoRankMem (2.00), NoEliteRef (2.50), and NoLayerSel (3.83). On the three ultra-high-dimensional datasets, CORAL also maintains the best average HV rank (1.67), whereas NoRankMem and NoEliteRef both obtain 2.17 and NoLayerSel degrades to 4.00. After removing stratified environmental selection, the HV values decrease on multiple datasets. This suggests that relying only on non-dominated sorting and crowding distance is insufficient to maintain the distribution of solutions across different feature-size regions. By limiting the excessive concentration of candidate solutions

in a single sparsity region, stratified environmental selection helps improve the coverage ability and distribution balance of the non-dominated solution set.

Table 9. HV comparison of CORAL and its ablated variants on 18 benchmark datasets. The best result in each row is shown in bold with a gray background. The $+/-/=$ row summarizes the number of datasets on which each ablated variant is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	NoLayerSel	NoRankMem	NoEliteRef	CORAL
1	**8.89×10^{-1}** (0.00×10^{0})	8.71×10^{-1} (0.00×10^{0})	8.71×10^{-1} (0.00×10^{0})	8.58×10^{-1} (0.00×10^{0})
2	5.80×10^{-1} (6.65×10^{-3})	**6.66×10^{-1}** (5.83×10^{-3})	6.40×10^{-1} (2.24×10^{-2})	6.59×10^{-1} (1.00×10^{-2})
3	7.50×10^{-1} (2.95×10^{-2})	9.50×10^{-1} (5.65×10^{-4})	9.09×10^{-1} (2.68×10^{-3})	**9.86×10^{-1}** (9.60×10^{-3})
4	7.96×10^{-1} (8.72×10^{-3})	9.61×10^{-1} (1.07×10^{-3})	9.49×10^{-1} (2.46×10^{-2})	**9.87×10^{-1}** (8.00×10^{-3})
5	7.29×10^{-1} (6.01×10^{-3})	9.13×10^{-1} (3.76×10^{-3})	9.01×10^{-1} (1.47×10^{-2})	**9.96×10^{-1}** (2.20×10^{-4})
6	5.32×10^{-1} (1.59×10^{-2})	8.18×10^{-1} (3.40×10^{-2})	8.55×10^{-1} (5.07×10^{-2})	**9.05×10^{-1}** (2.02×10^{-2})
7	6.46×10^{-1} (8.65×10^{-3})	**9.16×10^{-1}** (1.71×10^{-3})	8.85×10^{-1} (1.75×10^{-2})	8.96×10^{-1} (2.20×10^{-2})
8	6.21×10^{-1} (1.32×10^{-3})	**9.44×10^{-1}** (3.71×10^{-3})	9.34×10^{-1} (5.87×10^{-3})	9.36×10^{-1} (7.60×10^{-3})
9	6.41×10^{-1} (3.39×10^{-3})	**9.99×10^{-1}** (1.46×10^{-4})	**9.99×10^{-1}** (1.47×10^{-4})	9.47×10^{-1} (4.70×10^{-3})
10	5.54×10^{-1} (4.50×10^{-2})	**1.00×10^{0}** (1.36×10^{-16})	**1.00×10^{0}** (1.36×10^{-16})	9.99×10^{-1} (3.70×10^{-4})
11	6.08×10^{-1} (1.08×10^{-3})	9.99×10^{-1} (0.00×10^{0})	9.99×10^{-1} (0.00×10^{0})	**1.00×10^{0}** (1.40×10^{-4})
12	5.51×10^{-1} (1.37×10^{-4})	9.99×10^{-1} (1.96×10^{-4})	9.99×10^{-1} (8.51×10^{-5})	**1.00×10^{0}** (6.80×10^{-5})
13	5.98×10^{-1} (8.67×10^{-4})	9.99×10^{-1} (9.95×10^{-5})	9.99×10^{-1} (1.36×10^{-16})	**1.00×10^{0}** (2.30×10^{-4})
14	5.52×10^{-1} (5.58×10^{-3})	9.83×10^{-1} (1.26×10^{-2})	9.68×10^{-1} (2.45×10^{-2})	**1.00×10^{0}** (1.30×10^{-4})
15	5.50×10^{-1} (9.95×10^{-3})	9.72×10^{-1} (1.49×10^{-4})	9.72×10^{-1} (1.18×10^{-4})	**9.98×10^{-1}** (1.10×10^{-2})
16	4.82×10^{-1} (2.36×10^{-3})	9.24×10^{-1} (2.63×10^{-2})	9.54×10^{-1} (6.69×10^{-5})	**9.69×10^{-1}** (2.40×10^{-4})
17	5.50×10^{-1} (3.78×10^{-3})	**9.99×10^{-1}** (7.29×10^{-5})	**9.99×10^{-1}** (2.19×10^{-5})	9.49×10^{-1} (2.20×10^{-2})
18	5.13×10^{-1} (6.72×10^{-3})	9.53×10^{-1} (1.50×10^{-2})	9.45×10^{-1} (1.33×10^{-4})	**1.00×10^{0}** (2.00×10^{-4})
$+/-/=$	1/17/0	7/11/0	4/14/0	–

The results of NoRankMem show that removing rank memory decreases HV on most datasets, indicating that feature-occurrence information from elite solutions provides useful search guidance. Compared with NoRankMem, the complete CORAL framework wins on 11 datasets and loses on seven datasets, with an average HV improvement of approximately 0.0122. This comparatively moderate degradation suggests that rank memory mainly strengthens search directionality rather than dominating the entire search behavior. However, NoRankMem remains competitive on a few datasets, suggesting that the benefit of memory guidance may depend on the redundancy structure and the distribution of discriminative feature combinations.

The ablation results of elite local refinement show that local correction around high-quality solutions can improve solution-set quality on some datasets. Compared with NoEliteRef, the complete CORAL framework wins on 14 datasets and loses on four datasets, with an average HV improvement of approximately 0.0171. This strategy generates re-

fined candidates through lightweight remove–add feature replacement, thereby providing additional local exploitation with limited computational cost.

Because refined candidates are evaluated together with the original elites and other offspring, they are retained only when they improve the objective trade-off or contribute to the stratified solution distribution. Therefore, the refinement step does not deterministically overwrite rare but useful features; instead, it offers an additional local search opportunity whose effects remain filtered by objective evaluation and environmental selection. This also explains why the improvement brought by elite refinement is moderate rather than dominant: the refinement is designed to provide additional local candidates, not to force high-frequency features into all elite subsets.

In summary, the ablation results provide quantitative evidence for the roles of the three mechanisms. Removing stratified environmental selection causes the largest average HV degradation, with CORAL outperforming NoLayerSel on 17 datasets and improving HV by about 0.3302 on average. This indicates that sparsity-aware distribution maintenance is the most influential component in the present setting. Removing rank memory and elite refinement also reduces HV on most datasets, with average HV improvements of approximately 0.0122 and 0.0171 for the complete CORAL framework, respectively. These results suggest that rank memory mainly contributes to search guidance, while elite refinement provides additional local exploitation around promising solutions.

5.5. Additional Cross-Classifier External Evaluation

To examine the robustness of the selected feature subsets across different classifiers, an additional external evaluation was conducted using a linear SVM on the same eight representative datasets. It should be noted that the linear SVM was not used during evolutionary search. For each independent run, the final non-dominated subsets obtained by each algorithm were re-evaluated using the external linear SVM model on the held-out test set, and the selected-feature ratio and SVM test error were used as the two objectives to compute HV.

As shown in Table 10, CORAL achieves the best average external-SVM HV rank among the compared algorithms and obtains the best mean HV on DLBCL, ALLAML, and Arcene_8k. These results suggest that the feature subsets obtained by CORAL are not solely dependent on the weighted KNN wrapper used during search but can also maintain competitive performance under a different classifier. Since the external SVM HV values are computed from the final non-dominated solution sets rather than from post hoc selection of a single test-best subset, this analysis provides additional evidence for addressing the concern that the reported performance is mainly driven by classifier-specific bias.

Table 10. HV comparison of different algorithms under external support vector machine (SVM) evaluation on eight representative datasets. The best result in each row is shown in bold with a gray background. The symbols $+$, $-$, and $=$ indicate that the corresponding algorithm is significantly better than, significantly worse than, or not significantly different from CORAL, respectively, at $\alpha = 0.05$.

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
4	6.8212×10^{-1}	**8.0849×10^{-1}**	6.3841×10^{-1}	7.1195×10^{-1}	7.7769×10^{-1}	7.5971×10^{-1}	7.7807×10^{-1}
	(6.24×10^{-2}) $-$	(5.51×10^{-2}) $=$	(1.15×10^{-1}) $-$	(6.88×10^{-2}) $-$	(6.29×10^{-2}) $=$	(9.05×10^{-2}) $=$	(6.79×10^{-2})
11	1.8025×10^{-1}	9.1840×10^{-1}	8.7860×10^{-1}	2.1622×10^{-1}	4.4507×10^{-1}	9.3458×10^{-1}	**9.3699×10^{-1}**
	(2.18×10^{-2}) $-$	(1.41×10^{-1}) $=$	(1.52×10^{-1}) $=$	(3.46×10^{-2}) $-$	(3.77×10^{-2}) $-$	(1.06×10^{-1}) $=$	(8.90×10^{-2})
12	1.0805×10^{-1}	**8.9776×10^{-1}**	8.5423×10^{-1}	1.2551×10^{-1}	3.4793×10^{-1}	8.4205×10^{-1}	8.8737×10^{-1}
	(1.17×10^{-2}) $-$	(9.57×10^{-2}) $=$	(1.06×10^{-1}) $=$	(2.01×10^{-2}) $-$	(3.51×10^{-2}) $-$	(1.95×10^{-1}) $=$	(7.59×10^{-2})
13	1.6370×10^{-1}	9.2358×10^{-1}	8.7854×10^{-1}	1.9085×10^{-1}	3.9460×10^{-1}	7.8155×10^{-1}	**9.4171×10^{-1}**
	(2.62×10^{-2}) $-$	(1.15×10^{-1}) $=$	(1.53×10^{-1}) $=$	(2.78×10^{-2}) $-$	(5.81×10^{-2}) $-$	(2.96×10^{-1}) $=$	(1.07×10^{-1})

Table 10. *Cont.*

NO	HEA	MFFS	MOEA-NZD	MOEA/D	NSGA-II	PRDH	CORAL
14	1.2422×10^{-1} (9.90×10^{-3}) −	8.1991×10^{-1} (6.31×10^{-2}) −	6.8797×10^{-1} (1.27×10^{-1}) −	1.1708×10^{-1} (1.59×10^{-2}) −	3.1779×10^{-1} (2.62×10^{-2}) −	7.8619×10^{-1} (7.54×10^{-2}) −	$\mathbf{8.6879 \times 10^{-1}}$ (7.80×10^{-2})
15	1.0569×10^{-1} (8.76×10^{-3}) −	$\mathbf{9.3199 \times 10^{-1}}$ (4.17×10^{-2}) =	7.6869×10^{-1} (6.45×10^{-2}) −	1.1542×10^{-1} (1.56×10^{-2}) −	3.0594×10^{-1} (2.48×10^{-2}) −	9.0732×10^{-1} (4.18×10^{-2}) =	9.2281×10^{-1} (3.65×10^{-2})
16	1.2735×10^{-1} (1.56×10^{-2}) −	8.7602×10^{-1} (9.99×10^{-2}) =	8.3923×10^{-1} (1.28×10^{-1}) =	1.3938×10^{-1} (2.55×10^{-2}) −	3.2842×10^{-1} (2.88×10^{-2}) −	$\mathbf{9.0276 \times 10^{-1}}$ (1.75×10^{-1}) +	8.9550×10^{-1} (7.60×10^{-2})
17	1.2511×10^{-1} (1.11×10^{-2}) −	8.9823×10^{-1} (9.87×10^{-2}) =	8.6848×10^{-1} (1.13×10^{-1}) =	1.3563×10^{-1} (2.26×10^{-2}) −	3.2395×10^{-1} (3.44×10^{-2}) −	$\mathbf{9.7949 \times 10^{-1}}$ (4.89×10^{-2}) +	9.5281×10^{-1} (6.30×10^{-2})
+/−/=	0/8/0	0/1/7	0/3/5	0/8/0	0/7/1	2/1/5	−
Rank	6.750	2.000	4.125	6.000	4.750	2.750	**1.625**

5.6. Stability and Parameter Sensitivity Analysis

To further examine the stability of the selected feature subsets, the representative solutions chosen by minimum training cross-validation error were compared across independent runs on the same eight representative datasets used in the external SVM validation. Table 11 reports both the mean pairwise Jaccard similarity and an adjusted Kuncheva-style stability index that accounts for expected random overlap, together with the mean and standard deviation of external linear SVM Macro-F_1 across runs. The two stability indices show a consistent pattern, with mean values mostly ranging from about 0.60 to 0.75 across the selected datasets. At the same time, the cross-run Macro-F_1 deviations remain moderate, indicating that CORAL tends to identify relatively consistent sparse subsets while maintaining stable external predictive performance. These results suggest that, on the representative datasets considered here, the selected subsets are not only discriminative under an external classifier but also reasonably repeatable across independent runs.

Table 11. Subset stability and cross-run external SVM consistency of CORAL on eight representative datasets. Gray backgrounds indicate the best value for each metric.

NO	Mean Jaccard	Mean Kuncheva	Macro-F_1 (Mean ± Std)
4	0.6368	0.6346	0.7528 ± 0.0461
11	0.6023	0.6022	0.9075 ± 0.0831
12	0.7494	0.7494	0.8942 ± 0.0681
13	0.6161	0.6160	0.9473 ± 0.0688
14	0.7126	0.7126	0.8169 ± 0.0634
15	0.6437	0.6436	0.8487 ± 0.0830
16	0.7034	0.7034	0.9037 ± 0.0669
17	0.7540	0.7540	0.9212 ± 0.0692

The sensitivity of the main CORAL parameters was examined on the same eight representative datasets used in the external SVM validation. A one-factor-at-a-time protocol was adopted: when one parameter was varied, the other parameters were fixed to their default values. The tested parameters included the number of sparsity layers B, the number of refined elites N_m, the memory update coefficient ρ, and the memory-guidance strength γ. Table 12 summarizes the average HV ranks and the number of best cases obtained by different parameter settings.

As shown in Table 12, $B = 5$ and $B = 4$ obtain the same average HV rank, while $B = 3$ performs worse. Although $B = 4$ obtains one more best case, $B = 5$ provides a finer sparsity stratification and was therefore used as the default setting. For elite local refinement, $N_m = 3$ achieves the best average rank, indicating that refining a small number of elite solutions is sufficient and avoids excessive local exploitation. For the memory update coefficient, $\rho = 0.4$ slightly outperforms $\rho = 0.2$ and $\rho = 0.1$, suggesting that a moderate contribution from the current elite frequency is beneficial. For the memory-guidance strength, $\gamma = 0.4$ obtains the best average rank, whereas larger values are less

stable. This result indicates that rank memory is useful as a soft search bias, but overly strong memory guidance may reduce exploration. Based on these results, CORAL used $B = 5$, $N_m = 3$, $\rho = 0.4$, and $\gamma = 0.4$ in the comparative experiments.

Table 12. Summary of HV sensitivity analysis of CORAL on eight representative datasets. The average rank is computed over the eight datasets, and a smaller rank indicates better performance. The values in each row are listed in the same order as the tested values.

Parameter	Tested Values	Average HV Rank	Best Cases	Selected Value
B	5/4/3	1.750/1.750/2.500	3/4/1	5
N_m	3/5/10	1.750/2.000/2.250	4/2/2	3
ρ	0.4/0.2/0.1	1.875/2.000/2.125	4/1/3	0.4
γ	0.4/0.6/0.8	1.625/2.000/2.375	4/3/1	0.4

6. Conclusions

This paper proposed CORAL, a rank-memory search framework for high-dimensional MOFS. CORAL uses a joint score–cardinality representation with Top-K decoding to model feature priorities and subset sizes. Rank memory provides historical feature-preference guidance, while elite local refinement and stratified environmental selection support local candidate correction and sparsity-aware solution maintenance.

Experiments on 18 benchmark datasets showed that CORAL achieved competitive solution-set quality while maintaining reasonable test classification performance, feature compactness, and computational efficiency. Compared with representative MOFS algorithms, CORAL emphasized the trade-off among classification performance, feature compression, and solution-set distribution rather than only minimizing the number of selected features. Additional external linear SVM evaluation, subset stability analysis, parameter sensitivity analysis, and ablation results further supported the robustness and complementary effects of the proposed mechanisms.

Several limitations remain. The current rank-memory mechanism is mainly based on feature occurrence frequencies and does not explicitly model high-order feature interactions. The numbers of sparsity layers and refined elites are also fixed. Hold-out-based testing may also be sensitive on extremely small-sample datasets, despite the use of repeated independent runs, controlled random seeds, and training-set cross-validation. Future work will consider nested cross-validation or repeated resampling protocols to provide more conservative performance estimates on such datasets. Future work will investigate feature-interaction modeling, adaptive memory updating, and more flexible stratified control strategies.

Author Contributions: W.L.: Methodology, software, validation, formal analysis, investigation, data curation, visualization, and writing—original draft preparation. H.J.: Conceptualization, supervision, project administration, funding acquisition, resources, and writing—review and editing. C.H.: Investigation, validation, and writing—review and editing. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Natural Science Foundation of Fujian Province, grant number 2025J011049.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets used in this study are public benchmark datasets. The MATLAB implementation of CORAL is publicly available at: https://github.com/1434413686adads-cmyk/CORAL-MOFS (accessed on 9 June 2026). Additional implementation details are available from the corresponding author upon reasonable request.

Acknowledgments: The authors gratefully acknowledge the support provided by the Fujian Key Laboratory of Agricultural IoT Applications and the IoT Application Engineering Research Center of Colleges and Universities in Fujian Province.

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Bolón-Canedo, V.; Sánchez-Maroño, N.; Alonso-Betanzos, A. Recent advances and emerging challenges of feature selection in the context of big data. *Knowl.-Based Syst.* **2015**, *86*, 33–45.
2. Bolón-Canedo, V.; Sánchez-Maroño, N.; Alonso-Betanzos, A. Feature selection for high-dimensional data. *Prog. Artif. Intell.* **2016**, *5*, 65–75.
3. Dhal, P.; Azad, C. A comprehensive survey on feature selection in the various fields of machine learning. *Appl. Intell.* **2022**, *52*, 4543–4581.
4. Velliangiri, S.; Alagumuthukrishnan, S.; Thankumar Joseph, S.I. A review of dimensionality reduction techniques for efficient computation. *Procedia Comput. Sci.* **2019**, *165*, 104–111.
5. Jia, W.; Sun, M.; Lian, J.; Hou, S. Feature dimensionality reduction: A review. *Complex Intell. Syst.* **2022**, *8*, 2663–2693.
6. Guyon, I.; Elisseeff, A. An introduction to variable and feature selection. *J. Mach. Learn. Res.* **2003**, *3*, 1157–1182.
7. Li, J.; Cheng, K.; Wang, S.; Morstatter, F.; Trevino, R.P.; Tang, J.; Liu, H. Feature selection: A data perspective. *ACM Comput. Surv.* **2017**, *50*, 1–45.
8. Labani, M.; Moradi, P.; Ahmadizar, F.; Jalili, M. A novel multivariate filter method for feature selection in text classification problems. *Eng. Appl. Artif. Intell.* **2018**, *70*, 25–37.
9. Abdollahzadeh, B.; Gharehchopogh, F.S. A multi-objective optimization algorithm for feature selection problems. *Eng. Comput.* **2022**, *38*, 1845–1863.
10. Yue, X.; Liao, Y.; Peng, H.; Kang, L.; Zeng, Y. A high-dimensional feature selection algorithm via fast dimensionality reduction and multi-objective differential evolution. *Swarm Evol. Comput.* **2025**, *94*, 101899.
11. Deb, K.; Pratap, A.; Agarwal, S.; Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* **2002**, *6*, 182–197.
12. Zhang, Q.; Li, H. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Trans. Evol. Comput.* **2007**, *11*, 712–731.
13. Li, G.; Chen, Z.; Zhou, Y.; Huang, Z.; Xia, X. Towards large-scale multi-objective feature selection: A two-stage evolutionary algorithm guided by dual feature weightings. *Expert Syst. Appl.* **2026**, *298*, 129823.
14. Huang, J.; Chen, Y.; Heidari, A.A.; Liu, L.; Chen, H.; Liang, G. DPDEPSO: A particle swarm optimization for balancing different objectives in multi-objective feature selection. *Expert Syst. Appl.* **2025**, *293*, 128709.
15. Yue, X.; Zuo, X.; Ling, P.; Xiong, C.; Peng, H.; Zeng, Y. High-dimensional multi-objective feature selection with niche-based binary differential evolution. *Expert Syst. Appl.* **2026**, *298*, 129478.
16. Namakin, M.; Rouhani, M.; Sabzekar, M. A multi-objective feature selection method considering the interaction between features. *Inf. Syst. Front.* **2025**, *27*, 925–940.
17. Chen, C.; Yao, X.; Gong, D.; Tu, H. A multi-objective evolutionary algorithm for feature selection incorporating dominance-based initialization and duplication analysis. *Swarm Evol. Comput.* **2025**, *95*, 101914.
18. Wang, M.; Ge, H.; Wang, X.; Sun, L.; Hou, Y.; Li, B. An evolutionary multitasking algorithm for multi-objective feature selection using dual-perspective reduction. *Eng. Appl. Artif. Intell.* **2025**, *152*, 110764.
19. Bidgoli, A.A.; Rahnamayan, S. Multi-phase evolutionary search space shrinking for large-scale multi-objective feature selection. *Appl. Soft Comput.* **2025**, *184*, 113755.
20. Yu, X.; Hu, Z.; Luo, W.; Xue, Y. Reinforcement learning-based multi-objective differential evolution algorithm for feature selection. *Inf. Sci.* **2024**, *661*, 120185.
21. Chiregi, M.; Mazinani, M.; Mirzarezaee, M. A new feature selection method using deep learning and graph representation in high-dimensional datasets. *Knowl.-Based Syst.* **2025**, *329*, 114338.
22. Coello Coello, C.A.; Lamont, G.B.; Van Veldhuizen, D.A. *Evolutionary Algorithms for Solving Multi-Objective Problems*; Springer: Berlin/Heidelberg, Germany, 2007.
23. Xue, B.; Fu, W.; Zhang, M. Multi-objective feature selection in classification: A differential evolution approach. In *Proceedings of the Asia-Pacific Conference on Simulated Evolution and Learning, Dunedin, New Zealand, 15–18 December 2014*; Springer: Berlin/Heidelberg, Germany, 2014; pp. 516–528.

24. Matharaarachchi, S.; Domaratzki, M.; Muthukumarana, S. Assessing feature selection method performance with class imbalance data. *Mach. Learn. Appl.* **2021**, *6*, 100170.
25. Hu, P.; Zhu, J. A filter-wrapper model for high-dimensional feature selection based on evolutionary computation. *Appl. Intell.* **2025**, *55*, 581.
26. Cherrington, M.; Xu, Q.; Thabtah, F.; Lu, J. Feature selection: Filter methods performance challenges. In *Proceedings of the 2019 International Conference on Computer and Information Sciences (ICCIS), Sakaka, Saudi Arabia, 3–4 April 2019*; IEEE: New York, NY, USA, 2019; pp. 1–4.
27. Asim, M.N.; Wasim, M.; Ali, M.S.; Rehman, A. Comparison of feature selection methods in text classification on highly skewed datasets. In *Proceedings of the 2017 First International Conference on Latest Trends in Electrical Engineering and Computing Technologies (INTELLECT), Karachi, Pakistan, 15–16 November 2017*; IEEE: New York, NY, USA, 2017; pp. 1–8.
28. Wang, A.; An, N.; Chen, G.; Li, L.; Alterovitz, G. Accelerating wrapper-based feature selection with K-nearest-neighbor. *Knowl.-Based Syst.* **2015**, *83*, 81–91.
29. Zhang, J.; Xiong, Y.; Min, S. A new hybrid filter/wrapper algorithm for feature selection in classification. *Anal. Chim. Acta* **2019**, *1080*, 43–54.
30. Taha, Z.Y.; Abdullah, A.A.; Rashid, T.A. Optimizing feature selection with genetic algorithms: A review of methods and applications. *Knowl. Inf. Syst.* **2025**, *67*, 9739–9778.
31. Shen, C.; Zhang, K. Two-stage improved grey wolf optimization algorithm for feature selection on high-dimensional classification. *Complex Intell. Syst.* **2022**, *8*, 2769–2789.
32. Hamdani, T.M.; Won, J.M.; Alimi, A.M.; Karray, F. Multi-objective feature selection with NSGA-II. In *Proceedings of the International Conference on Adaptive and Natural Computing Algorithms, Warsaw, Poland, 11–14 April 2007*; Springer: Berlin/Heidelberg, Germany, 2007; pp. 240–247.
33. Jiao, R.; Xue, B.; Zhang, M. Benefiting from single-objective feature selection to multiobjective feature selection: A multiform approach. *IEEE Trans. Cybern.* **2022**, *53*, 7773–7786.
34. Liang, J.; Zhang, Y.; Qu, B.; Chen, K.; Yu, K.; Yue, C. A multiform optimization framework for multiobjective feature selection in classification. *IEEE Trans. Evol. Comput.* **2023**, *28*, 1024–1038.
35. Liu, W.; Ling, Q.; Han, F.; Han, H.; Shi, J. Feature selection based on multimodal multi-objective particle swarm optimization and prior information. *Pattern Anal. Appl.* **2025**, *28*, 181.
36. Majidpour, J.; Beitollahi, H. Multi-objective feature selection of radiomics and deep learning features for breast cancer subtype detection: J. Majidpour and H. Beitollahi. *J. Supercomput.* **2025**, *81*, 1330.
37. Verma, G.; Sahu, T.P. PSI-MFS: lightweight multi-objective feature selection for enhanced multi-label classification: G. Verma et al. *J. Supercomput.* **2025**, *81*, 755.
38. Li, S.; Zhang, K.; Chen, Q.; Wang, S.; Zhang, S. Feature selection for high dimensional data using weighted k-nearest neighbors and genetic algorithm. *IEEE Access* **2020**, *8*, 139512–139528.
39. Kumar, M.; Rath, N.K.; Swain, A.; Rath, S.K. Feature selection and classification of microarray data using MapReduce-based ANOVA and K-nearest neighbor. *Procedia Comput. Sci.* **2015**, *54*, 301–310.
40. Park, C.H.; Kim, S.B. Sequential random k-nearest neighbor feature selection for high-dimensional data. *Expert Syst. Appl.* **2015**, *42*, 2336–2342.
41. Wang, X.; Cheng, R.; Jin, Y. Sparse large-scale multiobjective optimization by identifying nonzero decision variables. *IEEE Trans. Syst. Man, Cybern. Syst.* **2024**, *54*, 6280–6292.
42. Liu, Z.; Han, F.; Ling, Q.; Han, H.; Jiang, J. A many-objective optimization evolutionary algorithm based on hyper-dominance degree. *Swarm Evol. Comput.* **2023**, *83*, 101411.
43. Jiao, R.; Xue, B.; Zhang, M. Solving multiobjective feature selection problems in classification via problem reformulation and duplication handling. *IEEE Trans. Evol. Comput.* **2022**, *28*, 846–860.
44. Shang, K.; Ishibuchi, H.; He, L.; Pang, L.M. A survey on the hypervolume indicator in evolutionary multiobjective optimization. *IEEE Trans. Evol. Comput.* **2020**, *25*, 1–20.
45. Jiang, S.; Zhang, J.; Ong, Y.S.; Zhang, A.N.; Tan, P.S. A simple and fast hypervolume indicator-based multiobjective evolutionary algorithm. *IEEE Trans. Cybern.* **2014**, *45*, 2202–2213.
46. Tian, Y.; Zhang, X.; Cheng, R.; Jin, Y. A multi-objective evolutionary algorithm based on an enhanced inverted generational distance metric. In *Proceedings of the 2016 IEEE Congress on Evolutionary Computation (CEC), Vancouver, BC, Canada, 24–29 July 2016*; IEEE: New York, NY, USA, 2016; pp. 5222–5229.
47. Sun, Y.; Yen, G.G.; Yi, Z. IGD indicator-based evolutionary algorithm for many-objective optimization problems. *IEEE Trans. Evol. Comput.* **2018**, *23*, 173–187.
48. Trawiński, B.; Smętek, M.; Telec, Z.; Lasota, T. Nonparametric statistical analysis for multiple comparison of machine learning regression algorithms. *Int. J. Appl. Math. Comput. Sci.* **2012**, *22*, 867–881.

49. Liu, J.; Xu, Y. T-Friedman test: A new statistical test for multiple comparison with an adjustable conservativeness measure. *Int. J. Comput. Intell. Syst.* **2022**, *15*, 29.
50. Tian, Y.; Cheng, R.; Zhang, X.; Jin, Y. PlatEMO: A MATLAB platform for evolutionary multi-objective optimization. *IEEE Comput. Intell. Mag.* **2017**, *12*, 73–87.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.